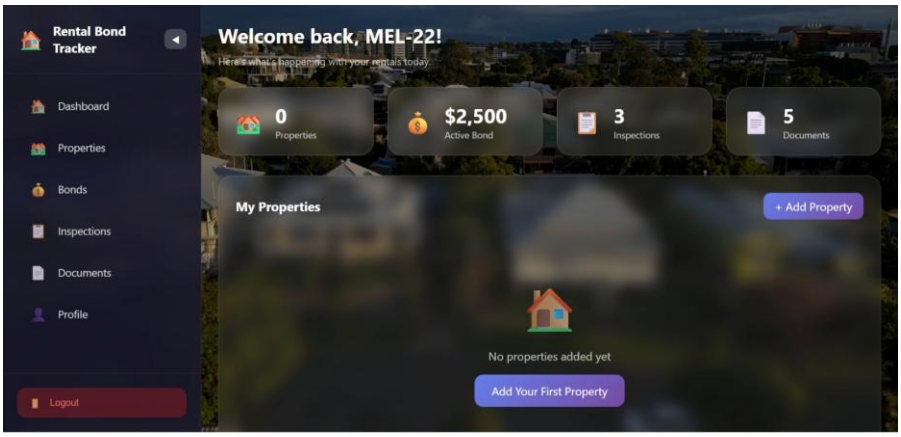


Software Requirements Specification (SRS) Report

Rental Bond Tracker

SUBMITTED BY
MEL-22

MR AAYUSH BHANDARI – K210839
MR DADHI RAM POUDEL – K231676 MR
MR SUJAN LAMICHHANE – K23119
MR NGUYEN NHAT LINH DINH – K221695
MR ASIM BHATTARAI – K240050
MR SHUBHAM SHRESTHA – K240901

SUBMITTED TO

MR SYED ALTAF

TABLE OF CONTENTS

1.Introduction …… 3
1.1.0 Background of study……. 3
1.1.1 Purpose of the Project……. 3
1.1.2 Objectives of the Project…… 4
1.1.3 Project Scope and Limitations…… 4-5
1.1.4 Project Assumptions……. 5-6
1.1.5 Company Profile……. 6
1.1.6 Organization Chart……. 7-8
1.1.7 Roles and Responsibilities…… 8-9
1.1.8 Key Deliverables…….. 9-12
1.1.9 Project Budget…… 12-15
1.1.10 Work Breakdown Structure (WBS) …… 16
1.1.11 Gantt Chart……. 17-18

2.1.1 Functional Requirements…… 19-38
2.1.2 Non-Functional Requirements (NFR) …… 38-43

3.1.1 System Architecture…… 43
3.1.2 Software and Hardware Architecture……. 43-48
3.1.3 Technology Stack…… 48-52

4.1.1 System Design – Balsamiq Cloud…… 53-57
4.1.2 UI Wireframes……. 57-62

5.1.1 Use Case Diagram……. 63-64
6.1.1 Context Diagram…… 64-66

7.1.1 DFD Level 1…… 67
7.1.2 DFD Level 2…… 68-74

8.1.1 Sequence Diagram…… 74-77

9.1.1 Entity Relationship Diagram (ERD) ……. 77-79
9.1.2 Data Dictionary……. 79-83

10.1.1 Class Diagram……. 84

11.1.1 Data Protection & Impact Assessment…….. 85-90

12.1.1 Test & Implementation Plan…….. 91-93

13.1.1 Responses to Feedback…….. 94

14.1.1 Reference…… 95

15.1.1 Appendix…… 96-100

1.INTRODUCTION

Rental Bond Tracker is a suggested web-based solution, which will assist tenants to control their rental bond payments, property inspections, and tenancy documents in a centralized online system. The following Software Requirements Specification (SRS) document includes the information on the business requirements, functional requirements, system behavior, and technical requirements that Rental Bond Tracker must have to ensure its successful development and implementation. The system will substitute manual record keeping systems like spreadsheets, email and paper files with an automated, secure and friendly web-based system to the tenant.

1.1.0 BACKGROUND OF THE PROJECT

In Australia, 30-35% of households rent their homes, and the average of rental bonds is 1,500-4,000 bonds, which depend on the property and the state. The present situation is that tenants have a lot of difficulties in handling their bond payments, inspection records and tenancy documentation. The records are usually mixed up in emails, text messages, paper files, and personal notes and may not be able to find them when they are most required especially at the end of a lease when bond refunds are determined.

The existing system of handling rental bonds is characterized by tenants having to keep a manual record of payment dates, store paper receipts, and keep different files of inspection reports. This disjointed methodology results in lost records, deadlines, and wrangles with landlords or property agents. In NSW alone, more than 10,000 bond disputes are taking place every year with tenants losing huge amounts of their bond as a result of missing evidence.

As the higher rental prices and more protections of tenants are required, a centralized digital system is required now. Rental Bond Tracker is a solution to these issues as it will offer a safer environment where tenants can store property information, confirm bond payments, keep inspection records, and all tenancy agreements in a single location.

1.1.1 PURPOSE OF THE PROJECT

The main intention of the Rental Bond Tracker is to ensure that the tenants have a unified, secure and easy-to-use system to administer the information of the rental bond and the documentation of tenancies. This platform aims to eradicate the manual record keeping inefficiencies by providing an orderly system of managing bond payments, storing lease agreements, and insurance of inspection records by tenants. This project will help to minimize bond disputes, enhance tenant confidence and enable quicker and simpler bond refunds by ensuring that important tenancy documentation is digitized, centrally located and readily available. The system is meant to enhance better organization of records, less administrative load on tenants and enhance predictability of bond outcome.

1.1.2 OBJECTIVE OF THE PROJECT

Functional Objectives:

- Design a user registration and authentication system in which tenants will be able to create accounts and log in safely.
- Add property management functionality that enables tenants to add, edit and delete property information such as landlord and agent information.
- Introduce bond tracking features that would allow tenants to keep track of bond amounts, payment dates, reference numbers and change bond status (Paid/Pending/Refunded).
- Offer management of inspection records where tenants can create an inspection record, add condition comments and post photo evidence.
- Provide document storage capability to enable uploading of lease agreements, bond receipts and other tenancy documents.
- Design a dashboard displaying bond summaries, upcoming inspection dates, and recent documents.

Technical Objectives:

- Use MD5 hash to encrypt passwords and store them in a secure encoding as required by the project.
- Make sure that data transmission is all encrypted on HTTPS.
- The system should be optimized to ensure that the pages are loaded in less than 3 seconds and the API responses in less than half a second.
- Implement an operational web application prototype that can be accessed by the use of the modern web browsers.

1.1.3 PROJECT SCOPE:

- User Management: registering users, logging in securely, updating profiles, changing passwords, resetting passwords by email, and deleting accounts
  Property Management: Add information about a property, like its address, landlord's name, agent's name, and contact numbers. Edit property information, and delete properties with confirmation.
- Bond Tracking: Keep track of the bond amount, payment date, reference number, link the bond to the property, update the bond status (Paid/Pending/Refunded), and see the bond history.
  Inspection Records: Make inspection entries with the date and property, add notes about the condition, upload photos as proof, and see the history of inspections.

- Document Management: You can upload lease agreements, receipts, and other files (PDF, JPEG, PNG, DOCX up to 10MB), link them to properties or bonds, and see and delete them.
- Dashboard: Show a summary of the bond, including its current status and amount, upcoming inspection dates, a list of recent documents, and buttons for quick actions.
- Security: MD5 password encryption, HTTPS transmission, session timeout after 30 minutes, and protection against SQL injection
- Data Storage: A PostgreSQL database with the right relationships and daily backups that happen automatically.

LIMITATION:

- Payment Processing: No way to connect bond payments to payment gateways.
- Mobile Apps: There are no native iOS or Android apps; only web-based ones.
- Portals for landlords and agents: There are no separate portals for landlords or property agents.
- Government Integration: There is no connection between government bond authorities and the RTBA.
- Email Notifications: There are no automated email reminders for inspections or bond refunds (this will be added in the future).
- Languages: Only support English
- APIs from other companies: No way to connect with property management systems outside of the company

1.1.4 PROJECT ASSUMPTION:

Stakeholder Assumptions:

- When filling property details and bond records, tenants will be honest in their information.
- The project supervisor will also offer feedback in time when it comes to requirement validation and testing.
- Skillup Labs will be the project sponsor and offer the needed support.

User-Related Assumptions:

- End users (tenants) are fairly computer literate and able to navigate a web application.
- The users will be guided on document uploads and data entry.
- Users have access to a computer or mobile device that is connected to the internet.

Technical Assumptions:

- Users have a modern web browser (Chrome, Firefox, Safari, Edge).

- The environment used will have adequate storage and processing capacity.
- The development environment (Node.js, PostgreSQL) is not going to change during the project.

Operational Assumptions:

- The team members will accomplish their tasks within the 12-week schedule.
- To monitor progress, team meetings will take place weekly.
- Before the deployment of the system, it will be thoroughly tested.

1.1.5 COMPANY PROFILE:

Name of the company: Skillup Labs

Industry : Professional Development and Education Technology

Description of the Company:

Australia-based Skillup Labs is a company that specializes in professional development and educational technology. By giving students access to real-world project possibilities, the organization aims to close the gap between academic learning and industry experience. In collaboration with academic institutions, Skillup Labs provides industry-sponsored projects that let students use their technological expertise to address real-world business challenges.

Project Sponsor: Nabin Singh

Sponsor Role: Industry Project Provider

Contact Email: wil@skilluplabs.com.au

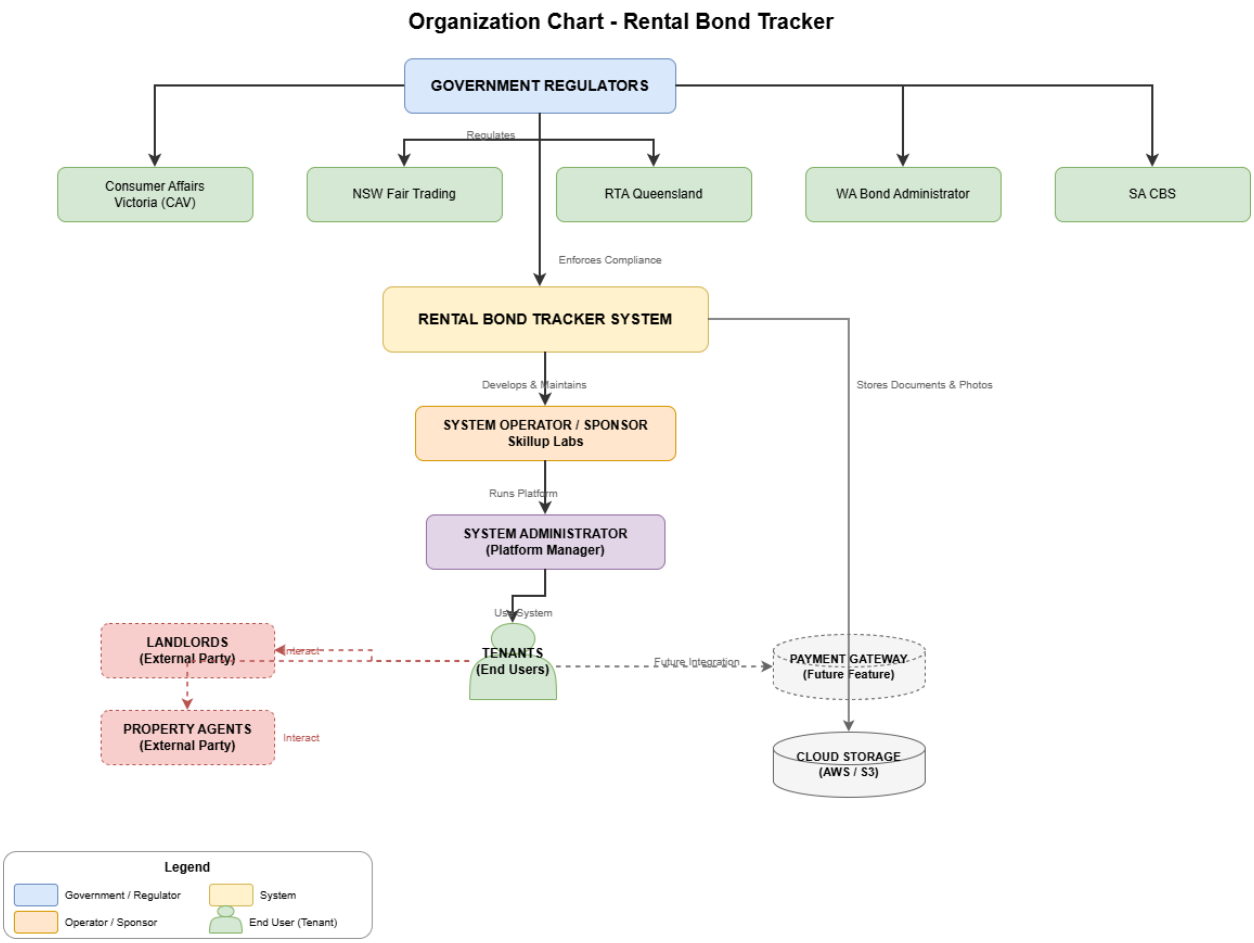
Industry Background:

Thirty to thirty-five percent of Australians rent, and there are about 2.5 million households in the country. Depending on the state and type of property, rental bonds often range from $1,500 to $4,000. Bond disputes are widespread; in NSW alone, over 10,000 cases are resolved each year. Tenants lack specialized tools to handle their own records because the majority of current solutions are aimed at landlords and property managers. By developing a tenant-centric platform that gives renters greater control over their tenancy information, this project fills this need.

1.1.6 ORGANIZATIONAL CHART

Organization Chart depicts the hierarchical setting of organizations, users and external parties that operate, govern, use and interact the Rental Bond Tracker system.

- Top Level - Government Regulators: Government regulators are at the peak and control the whole rental bond ecosystem. They establish tenancy laws and guard the rights of tenants in Australia.
- Second Level - State Bond Authorities: Under the federal level, each state has its bond authority such as Consumer Affairs Victoria, NSW Fair Trading, RTA Queensland, WA Bond Administrator and SA CBS. These authorities possess bond money, make refunds, deal with disputes and provide tenancy laws specific to states. To enforce compliance, they control the Rental Bond Tracker system.
- Third Level - The System: The Rental Bond Tracker system is in the middle. It is a tenant-focused service of bond tracking, document storage, and inspection records.
- Fourth Level - System Operator: Skill up Lab's sponsor and system operator. They build, operate, and store the platform, deliver compliance, and update and security.
- Fifth Level - System Administrator: The platform manager is in charge of day-to-day activities such as user management, data backup, security monitoring and troubleshooting of technical issues.
- Sixth Level – End users: The main users are tenants. Their dashboard allows them to open accounts, add properties, document the amount of the bond, upload documents, create inspection records, and monitor the status of the bond.
- External Parties - Landlords and Agents: The landlords get paid the bond and do inspections. This is because property agents handle properties on behalf of landlords. Both are interacted with by tenants and the system assists in recording these interactions.



1.1.7 ROLES AND RESPONSIBILITIES

Role	Name	Responsibility
Team Leader + Backend	Aayush Bhandari	Architecture, FRs/NFRs, backend APIs, feedback responses
Database Designer	Dadhi Ram Poudel	ERD, Data Dictionary, Class Diagram, SQL schema
Business Analyst	Sujan Lamichhane	Use Case, DFDs, Sequence Diagram, DPIA, Org Chart

Role	Name	Responsibility
Front-End Developer	Nguyen Nhat Linh Dinh	Balsamiq wireframes, UI components, API integration
Front-End + UI	Asim Bhattarai	Balsamiq wireframes, WBS, Gantt Chart, Budget, Test Plan
QA + AI	Shubham Shrestha	Test Plan, Implementation Plan, DPIA assist

1.1.8 KEY DELIVERABLES

(Application)

DELIVERABLE	DESCRIPTION	PRIORITY
Web Application	Fully functional Rental Bond Tracker (production-ready prototype)	High
User Registration & Login	Secure account creation and authentication with MD5 encryption	High
Property Management Module	Add, edit, delete, and view rental properties	High
Bond Tracking Module	Record bond amounts, payment dates, status (Paid/Pending/Refunded)	High

DELIVERABLE	DESCRIPTION	PRIORITY
Inspection Records Module	Create inspection entries, add condition notes, upload photos	High
Document Management Module	Upload, preview, download, and delete tenancy documents	High
Dashboard Module	Display bond summary, recent properties, upcoming inspections	High
Audit Trail Module	Log all user activities for security and accountability	Medium
User Profile Module	View and edit profile, change password	Medium
AI Chat Assistant	Help chatbot for tenant queries	Low

Documentation Deliverables

DOCUMENT	DESCRIPTION
Software Requirements Specification (SRS)	This document - complete requirements
System Architecture Documentation	3-tier architecture diagram and explanation
Database Schema	SQL schema for all tables
Entity Relationship Diagram (ERD)	Crow's foot notation diagram
Data Dictionary	All tables with column descriptions
Data Flow Diagrams (DFD)	Context, Level 0, Level 1, Level 2

DOCUMENT	DESCRIPTION
Class Diagram	UML class diagram with relationships
Use Case Diagram	System functionality from user perspective
Sequence Diagram	User registration and property flows
Test Plan	Test cases, testing strategy
Test Results	Results from testing phase
Implementation Plan	Deployment and integration steps
DPIA (Data Protection Impact Assessment)	Privacy risk assessment
User Manual / Quick Start Guide	How to use the application

Design Deliverables

DELIVERABLE	DESCRIPTION
UI Storyboard	Flow of screens showing user journey
Wireframes (Balsamiq Cloud)	Visual design of all major screens
Login Page Wireframe	Email, password, login button
Register Page Wireframe	Full name, email, password, phone

DELIVERABLE	DESCRIPTION
Dashboard Wireframe	Stats cards, quick actions, recent properties
Properties Page Wireframe	Property list with add/edit/delete
Add Property Form Wireframe	Address, landlord, agent, lease dates
Bonds Page Wireframe	Bond list and add bond form

1.1.9 PROJECT BUDGET

Rental Bond Tracker

CATEGORY	ITEM	COST (AUD)
Labor		
	Team Leader + Backend Developer (144 hours @ $60/hr)	$8,640
	Database Designer (120 hours @ $55/hr)	$6,600
	Business Analyst (120 hours @ $55/hr)	$6,600
	Front-End Developer (144 hours @ $55/hr)	$7,920
	Front-End + UI Designer (120 hours @ $55/hr)	$6,600
	QA + AI Engineer (96 hours @ $60/hr)	$5,760

CATEGORY	ITEM	COST (AUD)
Subtotal Labor		**$42,120**

SOFTWARE & TOOLS		
	Visual Studio Code (Enterprise)	$0 (Free)
	Node.js / PostgreSQL	$0 (Open source)
	GitHub Team (5 users)	$48/month × 3 months = $144
	Jira Software (Project Management)	$14/month × 3 months = $42
	Balsamiq Cloud (Team plan)	$12/month × 3 months = $36
	Postman Team Plan	$12/month × 3 months = $36
	Figma Professional	$15/month × 3 months = $45
Subtotal Software		**$303**

HOSTING & INFRASTRUCTURE (12 MONTHS)		
	Vercel Pro (Frontend)	$20/month × 12 = $240
	Railway (Backend + Database)	$20/month × 12 = $240
	AWS S3 (Document Storage - 50GB)	$15/month × 12 = $180
	CloudFront CDN	$10/month × 12 = $120
	Domain Name (.com.au)	$20/year
	SSL Certificate (Premium)	$200/year
	Email Service (SendGrid - 10,000 emails)	$15/month × 12 = $180
Subtotal Hosting		**$1,180**

TESTING & SECURITY		
	OWASP ZAP (Security testing)	$0 (Free)
	BrowserStack (Cross-browser testing)	$29/month × 3 months = $87

	Selenium Grid (Cloud)	$30/month × 3 months = $90
Subtotal Testing		**$177**

DOCUMENTATION & COMMUNICATION		
	Microsoft 365 Business (6 users)	$15/month × 6 × 3 months = $270
	Slack (Pro plan)	$8/month × 6 × 3 months = $144
Subtotal Communication		**$414**

TOTAL BUDGET:

Contingency (15%)		**$6,629**
GRAND TOTAL		**$50,823**

1.1.10 WORK BREAKDOWN STRUCTURE

1.0 Project Management (Aayush)

- 1.1 Team coordination
- 1.2 Milestone tracking
- 1.3 Lecturer communication

2.0 Requirements Analysis (Sujan+ Aayush)

- 2.1 Business requirements
- 2.2 Functional requirements
- 2.3 non-functional requirements

3.0 System Design (All)

- 3.1 Database design (Dadhi)
- 3.2 UI/UX design (Linh + Asim)
- 3.3 Architecture design (Aayush)

4.0 Development

- 4.1 Backend APIs (Dadhi + Aayush)
- 4.2 Frontend UI (Linh + Asim)
- 4.3 Integration (All)

5.0 Testing (Shubham+Aayush)

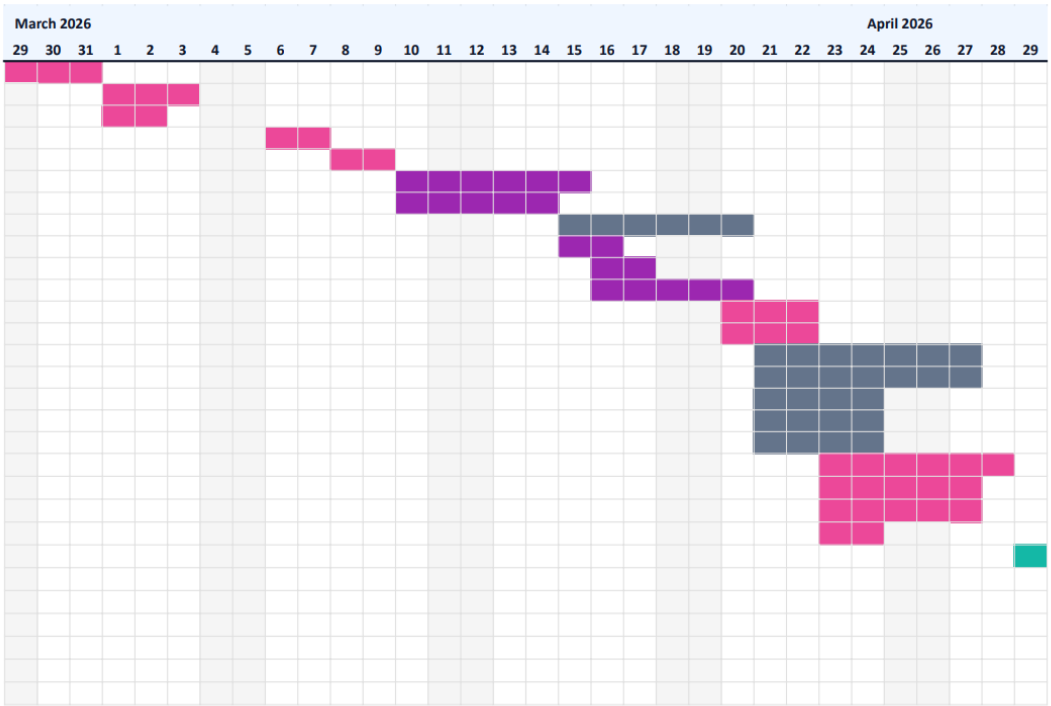
- 5.1 Unit testing
- 5.2 Integration testing
- 5.3 User acceptance testing

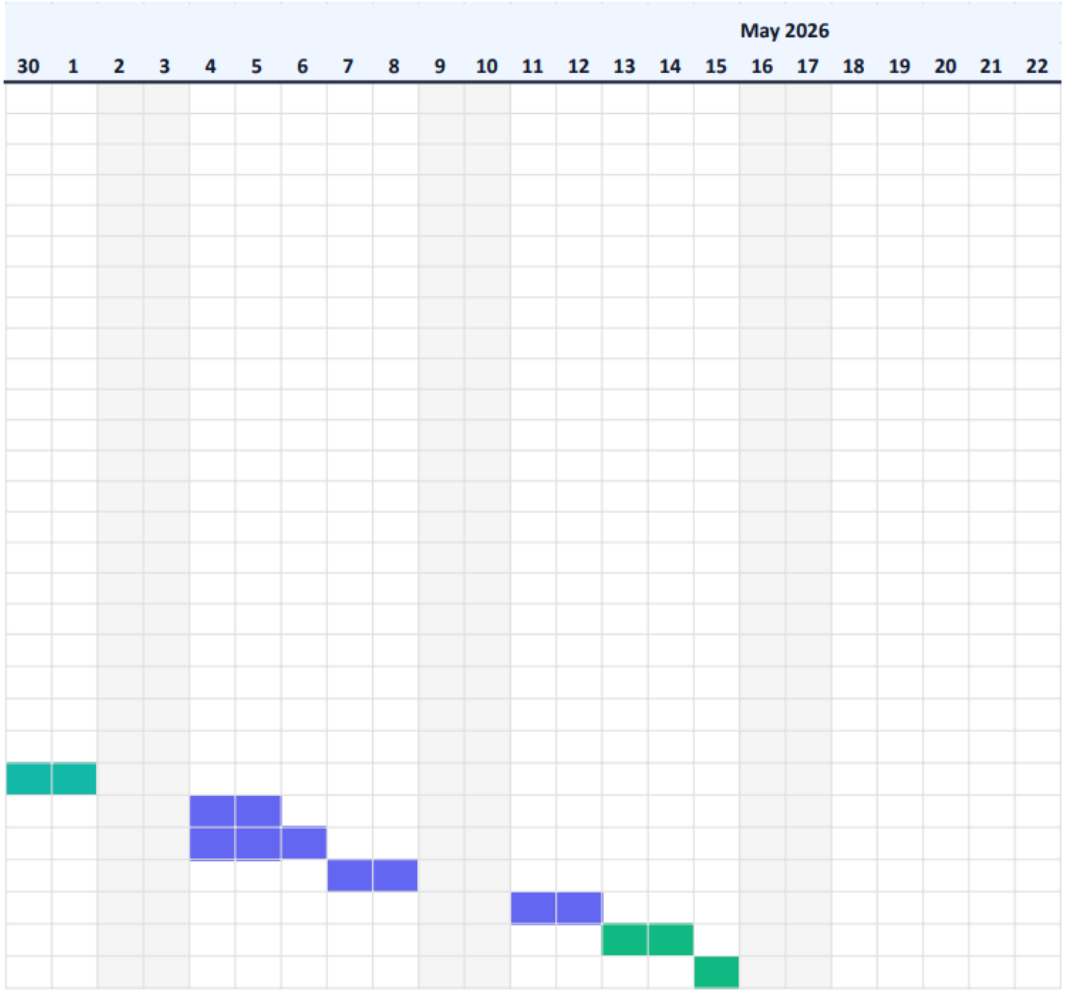
6.0 Deployment (Shubham)

- 6.1 Cloud deployment
- 6.2 Documentation(subham+Aayush)

1.1.11 GANTT CHART

Workstream	Task Name	Assignees	Start Date	End Date	Duration	16	17	18	19	20	21	22	23	28
Planning	Interview stakeholders for rental bor	Requirements Team	2026-03-23	2026-03-31	5									
Planning	Document functional requirements s	Requirements Team	2026-04-01	2026-04-03	3									
Planning	Define user personas and workflows	Requirements Team	2026-04-01	2026-04-02	2									
Planning	Create requirements traceability mat	Requirements Team	2026-04-06	2026-04-07	2									
Planning	Review and approve requirements w	Requirements Team	2026-04-08	2026-04-09	2									
Design	Design system architecture and com	Architecture Team	2026-04-10	2026-04-15	6									
Design	Design database schema and relatior	Database Team	2026-04-10	2026-04-14	5									
Backend Developn	Build bond tracking data models	Backend Team	2026-04-15	2026-04-20	6									
Design	Document database indexing strateg	Database Team	2026-04-15	2026-04-16	2									
Design	Create architecture documentation c	Architecture Team	2026-054-16	2026-04-17	2									
Design	Define API endpoints and data contr	Architecture Team	2026-04-16	2026-04-20	5									
Frontend Developr	Design frontend component library	Frontend Team	2026-04-20	2026-04-22	3									
Frontend Developr	Create user interface wireframes	Frontend Team	2026-04-22	2026-04-22	3									
Backend Developn	Implement user authentication mod	Backend Team	2026-04-21	2026-04-27	7									
Backend Developn	Develop payment processing backen	Backend Team	2026-04-21	2026-04-27	7									
Backend Developn	Create notification system endpoints	Backend Team	2026-04-21	2026-04-23	4									
Backend Developn	Implement rental bond calculation lo	Backend Team	2026-04-21	2026-04-23	4									
Backend Developn	Build admin dashboard API endpoint	Backend Team	2026-04-21	2026-04-23	4									
Frontend Developr	Develop responsive dashboard layou	Frontend Team	2026-04-23	2026-04-28	6									
Frontend Developr	Build bond tracking display compone	Frontend Team	2026-04-23	2026-04-27	5									
Frontend Developr	Implement payment form interface	Frontend Team	2026-04-23	2026-04-27	5									
Frontend Developr	Create notification settings interface	Frontend Team	2026-04-23	2026-04-23	2									
Integration	Integrate frontend with backend API:	Full Team	2026-04-29	2026-05-01	3									
Testing	Perform system integration testing	QA Team	2026-05-04	2026-05-05	2									
Testing	Execute functional test cases	QA Team	2026-05-04	2026-05-06	3									
Testing	Conduct performance and load testir	QA Team	2026-05-07	2026-05-08	2									
Testing	Conduct user acceptance testing ses	QA Team	2026-05-11	2026-05-12	2									
Deployment	Deploy application to production env	DevOps Team	2026-05-13	2026-05-14	2									
Deployment	Prepare and deliver final demo prese	Full Team	2026-05-15	2026-05-15	1									




May 2026
30
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22

2.1.1 FUNCTIONAL REQUIREMENTS - RENTAL BOND TRACKER

User account management (fr1-fr15)

FR No.	Functional Requirement	CRUD	Module	Comment
FR1	The system shall allow tenants to register a new account using email and password.	Create	User Account	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR2	The system shall allow tenants to log in with valid email and password.	Read	User Account	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR3	The system shall allow tenants to update their profile information (name, contact number, email).	Update	User Account	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR4	The system shall allow tenants to delete their account permanently with confirmation.	Delete	User Account	Ensures proper validation, role-based access, system integrity, and smooth user interaction.
FR5	The system shall allow tenants to reset forgotten	Update	User Account	Ensures proper validation, role-based access, system integrity, and smooth user

FR No.	Functional Requirement	CRUD	Module	Comment
	password via email recovery link.			interaction within this module.
FR6	The system shall allow tenants to change password after providing current password.	Update	User Account	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR7	The system shall enforce password strength (minimum 8 characters, alphanumeric).	Create	User Account	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR8	The system shall prevent duplicate email registration.	Create	User Account	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR9	The system shall allow tenants to view their profile information.	Read	User Account	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR10	The system shall allow tenants to upload profile picture.	Create	User Account	Ensures proper validation, role-based access, and smooth user interaction within this module.

FR No.	Functional Requirement	CRUD	Module	Comment
FR11	The system shall allow tenants to update profile picture.	Update	User Account	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR12	The system shall allow tenants to delete profile picture.	Delete	User Account	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR13	The system shall display last login date and time on dashboard.	Read	User Account	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR14	The system shall allow tenants to set preferred notification preferences.	Update	User Account	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR15	The system shall allow tenants to view account activity log.	Read	User Account	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.

Authentication and Authorization (FR16-FR25)

FR No.	Functional Requirement	CRUD	Module	Comment
FR16	The system shall encrypt user passwords using MD5 before storing.	Create	Authentication	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR17	The system shall verify user credentials during login using MD5 encryption.	Read	Authentication	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR18	The system shall invalidate session after 30 minutes of inactivity.	Update	Authentication	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR19	The system shall allow users to log out and terminate session.	Delete	Authentication	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR20	The system shall prevent access to	Read	Authentication	Ensures proper validation, role-based

FR No.	Functional Requirement	CRUD	Module	Comment
	protected pages without authentication.			access, system integrity, and smooth user interaction within this module.
FR21	The system shall allow only one active session per user account.	Create	Authentication	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR22	The system shall display session timeout warning 2 minutes before expiration.	Read	Authentication	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR23	The system shall allow users to extend session when warned.	Update	Authentication	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR24	The system shall lock account after 5 failed login attempts.	Update	Authentication	Ensures proper validation, role-based access, system integrity, and smooth user

FR No.	Functional Requirement	CRUD	Module	Comment
				interaction within this module.
FR25	The system shall allow account unlock via email verification.	Update	Authentication	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.

Property Management (FR26-FR40)

FR No.	Functional Requirement	CRUD	Module	Comment
FR26	The system shall allow tenants to add new property with address, landlord name, and agent name.	Create	Property	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR27	The system shall allow tenants to add landlord contact number and email.	Create	Property	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR28	The system shall allow tenants to add agent contact number and email.	Create	Property	Ensures proper validation, role-based access, system integrity, and smooth

FR No.	Functional Requirement	CRUD	Module	Comment
				user interaction within this module.
FR29	The system shall allow tenants to edit property details.	Update	Property	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR30	The system shall allow tenants to delete property with confirmation if no active bond exists.	Delete	Property	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR31	The system shall allow tenants to view all properties in list view.	Read	Property	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR32	The system shall allow tenants to search properties by address.	Read	Property	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR33	The system shall allow tenants to filter properties by status (active/previous).	Read	Property	Ensures proper validation, role-based access, system integrity, and smooth

FR No.	Functional Requirement	CRUD	Module	Comment
				user interaction within this module.
FR34	The system shall allow tenants to view property details page.	Read	Property	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR35	The system shall allow tenants to add multiple properties.	Create	Property	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR36	The system shall allow tenants to set property as current residence.	Update	Property	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR37	The system shall display lease start and end date for each property.	Create	Property	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR38	The system shall allow tenants to edit lease dates.	Update	Property	Ensures proper validation, role-based access, system integrity, and smooth

FR No.	Functional Requirement	CRUD	Module	Comment
				user interaction within this module.
FR39	The system shall allow tenants to add property photos.	Create	Property	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR40	The system shall allow tenants to view property photo gallery.	Read	Property	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.

Rental Bond Management (FR41-FR55)

FR No.	Functional Requirement	CRUD	Module	Comment
FR41	The system shall allow tenants to record bond amount for a property.	Create	Bond	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR42	The system shall allow tenants to record bond payment date.	Create	Bond	Ensures proper validation, role-based access, system integrity, and smooth user

FR No.	Functional Requirement	CRUD	Module	Comment
				interaction within this module.
FR43	The system shall allow tenants to record bond reference number.	Create	Bond	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR44	The system shall allow tenants to link bond to a specific property.	Create	Bond	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR45	The system shall allow tenants to update bond status to Paid.	Update	Bond	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR46	The system shall allow tenants to update bond status to Pending.	Update	Bond	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR47	The system shall allow tenants to update bond status to Refunded.	Update	Bond	Ensures proper validation, role-based access, system integrity, and smooth user

FR No.	Functional Requirement	CRUD	Module	Comment
				interaction within this module.
FR48	The system shall allow tenants to edit bond amount.	Update	Bond	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR49	The system shall allow tenants to edit bond payment date.	Update	Bond	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR50	The system shall allow tenants to view bond history for all properties.	Read	Bond	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR51	The system shall display bond amount, date, status, and property for each record.	Read	Bond	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR52	The system shall allow tenants to record refund amount when bond is refunded.	Create	Bond	Ensures proper validation, role-based access, system integrity, and smooth user

FR No.	Functional Requirement	CRUD	Module	Comment
				interaction within this module.
FR53	The system shall allow tenants to record refund date.	Create	Bond	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR54	The system shall calculate difference between bond amount and refund amount.	Read	Bond	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR55	The system shall allow tenants to delete bond record with confirmation.	Delete	Bond	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.

Inspection Records (FR56-FR70)

FR No.	Functional Requirement	CRUD	Module	Comment
FR56	The system shall allow tenants to create	Create	Inspection	Ensures proper validation, role-based access, system integrity, and smooth user

FR No.	Functional Requirement	CRUD	Module	Comment
	inspection record for a property.			interaction within this module.
FR57	The system shall allow tenants to select inspection date.	Create	Inspection	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR58	The system shall allow tenants to add condition notes for each room.	Create	Inspection	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR59	The system shall allow tenants to upload photos to inspection record.	Create	Inspection	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR60	The system shall allow tenants to upload multiple photos per inspection.	Create	Inspection	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR61	The system shall allow tenants to edit inspection notes.	Update	Inspection	Ensures proper validation, role-based access, system integrity, and smooth user

FR No.	Functional Requirement	CRUD	Module	Comment
				interaction within this module.
FR62	The system shall allow tenants to delete inspection photos.	Delete	Inspection	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR63	The system shall allow tenants to delete entire inspection record.	Delete	Inspection	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR64	The system shall display inspection history in chronological order.	Read	Inspection	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR65	The system shall allow tenants to filter inspections by property.	Read	Inspection	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR66	The system shall allow tenants to view inspection details page with photos.	Read	Inspection	Ensures proper validation, role-based access, system integrity, and smooth user

FR No.	Functional Requirement	CRUD	Module	Comment
				interaction within this module.
FR67	The system shall allow tenants to add entry inspection record (move-in).	Create	Inspection	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR68	The system shall allow tenants to add exit inspection record (move-out).	Create	Inspection	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR69	The system shall allow tenants to add routine inspection record.	Create	Inspection	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR70	The system shall allow tenants to rate property condition (1-5 scale).	Create	Inspection	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.

Document Management (FR71-FR85)

FR No.	Functional Requirement	CRUD	Module	Comment
FR71	The system shall allow tenants to upload lease agreement documents.	Create	Document	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR72	The system shall allow tenants to upload bond payment receipts.	Create	Document	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR73	The system shall allow tenants to upload inspection reports.	Create	Document	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR74	The system shall support PDF, JPEG, PNG, and DOCX file formats.	Create	Document	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR75	The system shall enforce maximum file size of 10MB per document.	Create	Document	Ensures proper validation, role-based access, system integrity, and smooth user

FR No.	Functional Requirement	CRUD	Module	Comment
				interaction within this module.
FR76	The system shall allow tenants to link documents to specific properties.	Create	Document	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR77	The system shall allow tenants to link documents to specific bonds.	Create	Document	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR78	The system shall allow tenants to add title and description to documents.	Create	Document	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR79	The system shall allow tenants to preview PDF and image files in browser.	Read	Document	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR80	The system shall display document list with file name, upload date, and property.	Read	Document	Ensures proper validation, role-based access, system integrity, and smooth user

FR No.	Functional Requirement	CRUD	Module	Comment
				interaction within this module.
FR81	The system shall allow tenants to download documents.	Read	Document	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR82	The system shall allow tenants to delete documents with confirmation.	Delete	Document	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR83	The system shall allow tenants to search documents by title or property.	Read	Document	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR84	The system shall allow tenants to sort documents by upload date.	Read	Document	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR85	The system shall display file type icon for each document.	Read	Document	Ensures proper validation, role-based access, system integrity, and smooth user

FR No.	Functional Requirement	CRUD	Module	Comment
				interaction within this module.

Dashboard and Summary (FR86-FR90)

FR No.	Functional Requirement	CRUD	Module	Comment
FR86	The system shall display dashboard showing current bond status and amount.	Read	Dashboard	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR87	The system shall display upcoming inspection dates on dashboard.	Read	Dashboard	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR88	The system shall display recent documents on dashboard.	Read	Dashboard	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.
FR89	The system shall display quick action buttons for common tasks (add	Read	Dashboard	Ensures proper validation, role-based access, system integrity, and smooth user

FR No.	Functional Requirement	CRUD	Module	Comment
	property, add inspection, upload document).			interaction within this module.
FR90	The system shall display total number of properties, bonds, inspections, and documents on dashboard.	Read	Dashboard	Ensures proper validation, role-based access, system integrity, and smooth user interaction within this module.

2.1.2 Non-Functional Requirements (NFR1 to NFR25)

NFR No.	Requirement	Category	Comment
NFR1	The system shall load all pages within 3 seconds under normal network conditions.	Performance	Ensures performance efficiency, security compliance, system stability, and a user-friendly experience.
NFR2	The system shall respond to API requests within 500 milliseconds for 95% of requests.	Performance	Ensures performance efficiency, security compliance, system stability, and a user-friendly experience.
NFR3	The system shall support up to 100 concurrent users	Scalability	Ensures performance efficiency, security compliance, system stability,

NFR No.	Requirement	Category	Comment
	without performance degradation.		and a user-friendly experience.
NFR4	The system shall encrypt all user passwords using MD5 before storing in database.	Security	Ensures performance efficiency, security compliance, system stability, and a user-friendly experience.
NFR5	The system shall transmit all data over HTTPS protocol only.	Security	Ensures performance efficiency, security compliance, system stability, and a user-friendly experience.
NFR6	The system shall prevent SQL injection attacks using parameterized queries.	Security	Ensures performance efficiency, security compliance, system stability, and a user-friendly experience.
NFR7	The system shall prevent cross-site scripting (XSS) attacks through input sanitization.	Security	Ensures performance efficiency, security compliance, system stability, and a user-friendly experience.
NFR8	The system shall be fully responsive and work on desktop, tablet, and mobile devices.	Usability	Ensures performance efficiency, security compliance, system stability,

NFR No.	Requirement	Category	Comment
			and a user-friendly experience.
NFR9	The system shall allow new users to complete core tasks within 5 minutes without training.	Usability	Ensures performance efficiency, security compliance, system stability, and a user-friendly experience.
NFR10	The system shall require no more than 3 clicks to access any major feature from the dashboard.	Usability	Ensures performance efficiency, security compliance, system stability, and a user-friendly experience.
NFR11	The system shall maintain 99.5% uptime excluding scheduled maintenance.	Reliability	Ensures performance efficiency, security compliance, system stability, and a user-friendly experience.
NFR12	The system shall perform automated database backups daily.	Reliability	Ensures performance efficiency, security compliance, system stability, and a user-friendly experience.
NFR13	The system shall allow data recovery within 4 hours from the last backup.	Reliability	Ensures performance efficiency, security compliance, system stability,

NFR No.	Requirement	Category	Comment
			and a user-friendly experience.
NFR14	The system shall support scaling to handle increased number of users without major code changes.	Scalability	Ensures performance efficiency, security compliance, system stability, and a user-friendly experience.
NFR15	The system shall include code comments for all major functions and complex logic.	Maintainability	Ensures performance efficiency, security compliance, system stability, and a user-friendly experience.
NFR16	The system shall store all source code in GitHub with complete commit history.	Maintainability	Ensures performance efficiency, security compliance, system stability, and a user-friendly experience.
NFR17	The system shall comply with Australian Privacy Principles (APPs) for data handling.	Compliance	Ensures performance efficiency, security compliance, system stability, and a user-friendly experience.
NFR18	The system shall permanently delete all user data upon account deletion request.	Compliance	Ensures performance efficiency, security compliance, system stability,

NFR No.	Requirement	Category	Comment
			and a user-friendly experience.
NFR19	The system shall support latest two versions of Chrome, Firefox, Safari, and Edge browsers.	Compatibility	Ensures performance efficiency, security compliance, system stability, and a user-friendly experience.
NFR20	The system shall display clear error messages for invalid user inputs.	Usability	Ensures performance efficiency, security compliance, system stability, and a user-friendly experience.
NFR21	The system shall log all user activities for audit purposes.	Security	Ensures performance efficiency, security compliance, system stability, and a user-friendly experience.
NFR22	The system shall maintain session timeout after 30 minutes of user inactivity.	Security	Ensures performance efficiency, security compliance, system stability, and a user-friendly experience.
NFR23	The system shall validate all form inputs before submission to ensure data integrity.	Reliability	Ensures performance efficiency, security compliance, system stability,

NFR No.	Requirement	Category	Comment
			and a user-friendly experience.
NFR24	The system shall be deployable on cloud platforms such as Vercel, AWS, or Railway.	Maintainability	Ensures performance efficiency, security compliance, system stability, and a user-friendly experience.
NFR25	The system shall provide user help documentation for all major features.	Usability	Ensures performance efficiency, security compliance, system stability, and a user-friendly experience.

3.1.1 SYSTEM ARCHITECTURE:

3.1.2 SOFTWARE ARCHITECTURE

The Rental Bond Tracker follows a **3-tier architecture**:

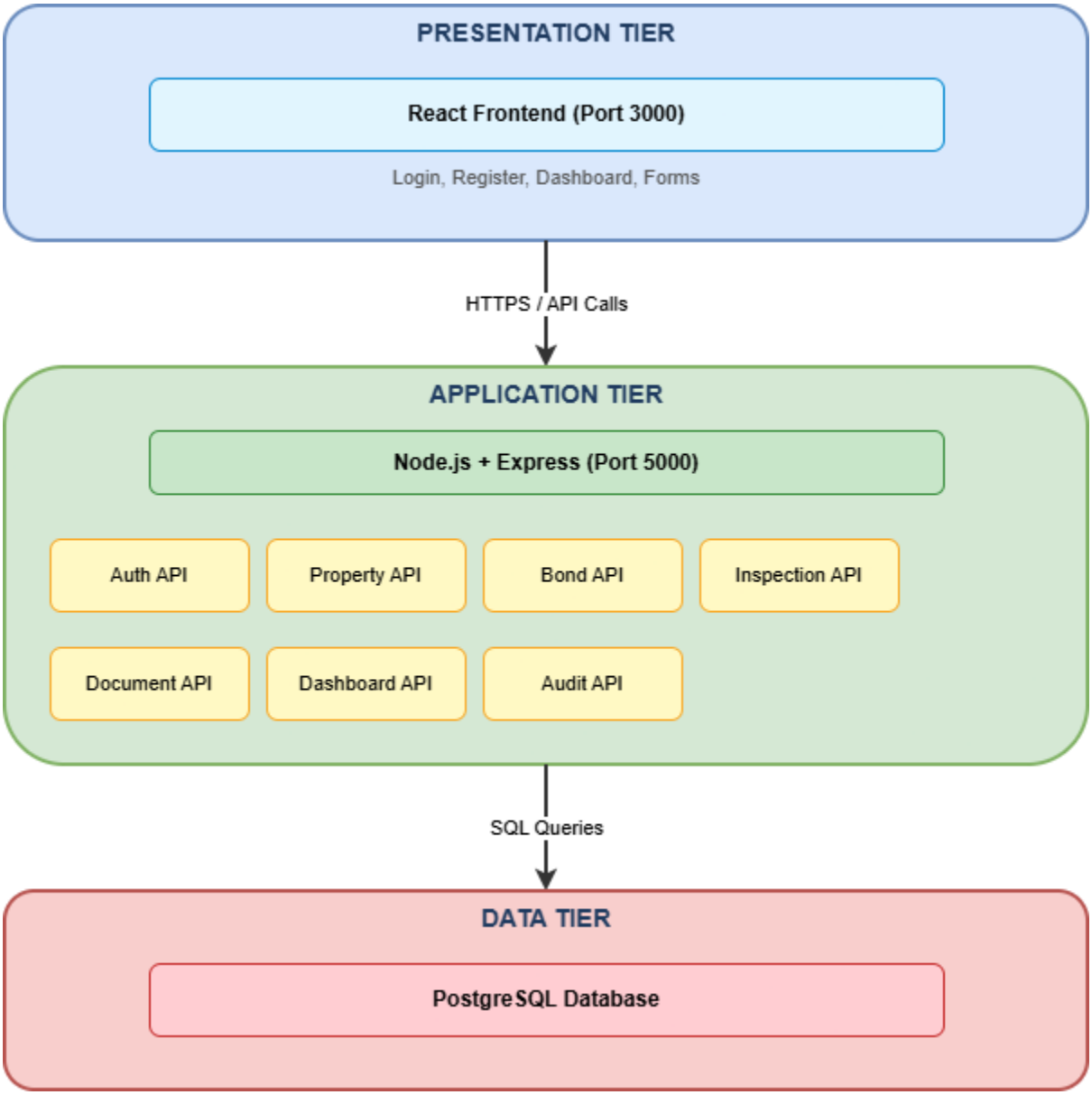
Tier	Component	Technology	Purpose
Presentation Tier	Frontend UI	React / Next.js	User interface, form handling, dashboard
Application Tier	Backend API	Node.js / Express	Business logic, authentication, file handling
Data Tier	Database	PostgreSQL	Data storage, queries, relationships

THREE TIERS EXPLAINED:

The initial level is the presentation layer/frontend. The tenant views and directly engages with this in his/her web browser. Developed in React and Next.js, the presentation tier manages all the user interface elements such as the login page, registration form, dashboard, property management forms, bond tracking screens, inspection record uploads and document management. The presentation layer receives the input of a tenant when he/she clicks a button or enters a form and transmits it as an HTTP request to the second layer. Notably, the frontend does not directly communicate with the database. It just communicates with the back end API. This level operates on the browser of the tenant, where the processing is done by their computer.

The second layer is the application layer also referred to as the backend or business logic layer. It is developed on Node.js and the Express framework and serves as the brain of the whole system. Once a request is received by the frontend, the application tier receives it and processes it by verifying the input data, confirming user permissions, implementing business rules and making a decision on what to retrieve or store. As an example, the application level will first ensure that the bond amount is a positive number, then confirm that the property ID is in the database, then encrypts any sensitive information such as passwords with MD5 hash and lastly makes a query to the third tier when a tenant provides a record of bond payment. The authentication of users is also processed at the application layer where active sessions are created as JSON Web Tokens (JWT) and the 30-minute time limit is implemented. This tier is based on cloud servers maintained by Railway and not the device of the tenant.

The third level is the data level that is the PostgreSQL database. This layer is responsible for permanently storing all information that the system needs to remember. The tables contained in the database are user, property, bond, inspection, document, and audit logs. In cases where the application layer requires access to the bond history of a tenant it issues a SQL query at the data layer, which, in turn, provides the requested records. The data tier implements data integrity by the use of primary key, foreign key and constraints. An example is that a bond record cannot exist without being connected to a valid property due to the foreign key constraint. The database is also ACID compliant that is, transactions are atomic, consistent, isolated, and durable. This level is also capable of running on cloud servers and is never visible to the front end or to the tenants.



3.1.2 HARDWARE ARCHITECTURE:

Development Environment

This is what each team member uses to write and test code.

COMPONENT	MINIMUM REQUIREMENT	RECOMMENDED
Processor	Intel i5 or AMD Ryzen 5	Intel i7 or AMD Ryzen 7
RAM	8 GB	16 GB
Storage	20 GB free space	256 GB SSD
Operating System	Windows 10 / macOS / Linux	Windows 11 / macOS latest
Network	Broadband internet	100 Mbps
Browser	Chrome, Firefox, or Edge	Latest version

Testing Environment

This environment matches production as closely as possible.

COMPONENT	SPECIFICATION
Processor	2 vCPU (virtual)
RAM	4 GB
Storage	50 GB SSD
Network	100 Mbps
Operating System	Ubuntu 22.04 LTS

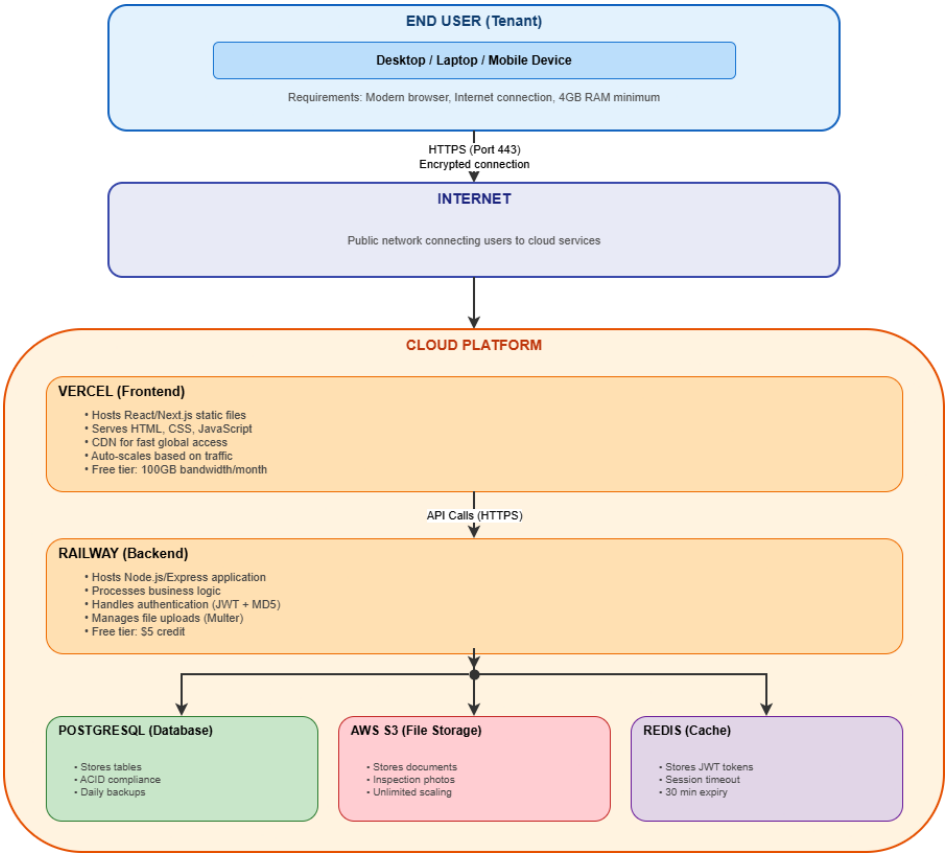
Production Environment (Cloud Hosting)

The system is deployed to cloud platforms rather than physical servers.

COMPONENT	SERVICE	SPECIFICATION
Frontend Hosting	Vercel	4 vCPU, auto-scaling
Backend Hosting	Railway	2 vCPU, 4 GB RAM
Database Hosting	Railway PostgreSQL	2 vCPU, 4 GB RAM, 50 GB storage
File Storage	AWS S3	50 GB standard storage

Development Hardware

Everyone in the development team needs a personal computer to code, test features and localize versions of the application. The development minimum is an Intel i5 or AMD Ryzen 5 processor with a laptop or desktop. This processor can compile React code, then can run Node.js servers, and database queries, without much slowdown. It should also have at least 8 gigabytes of RAM as the development environment has several processes such as Visual Studio Code, the React frontend development server, the Node.js backend server, and a local PostgreSQL instance running simultaneously. All these processes eat up memory and at a capacity of less than 8 gigabytes the computer would crawl. Storage needs are relatively small (20 gigabytes free space), but an SSD is highly recommended, as it loads files and the server can be started much faster than with a standard hard drive. The operating system may be Windows, macOS, or Linux as all development tools are cross-platform. To use GitHub, npm packages, and deploy to cloud services, a stable internet connection is needed, but real coding and testing can occur offline. To have a more enjoyable experience during development, the team will operate on recommended hardware instead of minimum requirements. Intel i7 or AMD Ryzen 7 processor is faster to compile especially with large React components. The 16 gigabytes of RAM will enable the use of other tools such as Docker containers or multiple web browser windows to test without drag. The project, node modules folders (which can be large) and several database backups can fit well on a 256 gigabyte SSD.



3.1.3 TECHNOLOGY STACK

The Rental Bond Tracker is built using modern, open-source technologies which are free as well as documented and also widely used in the industry.

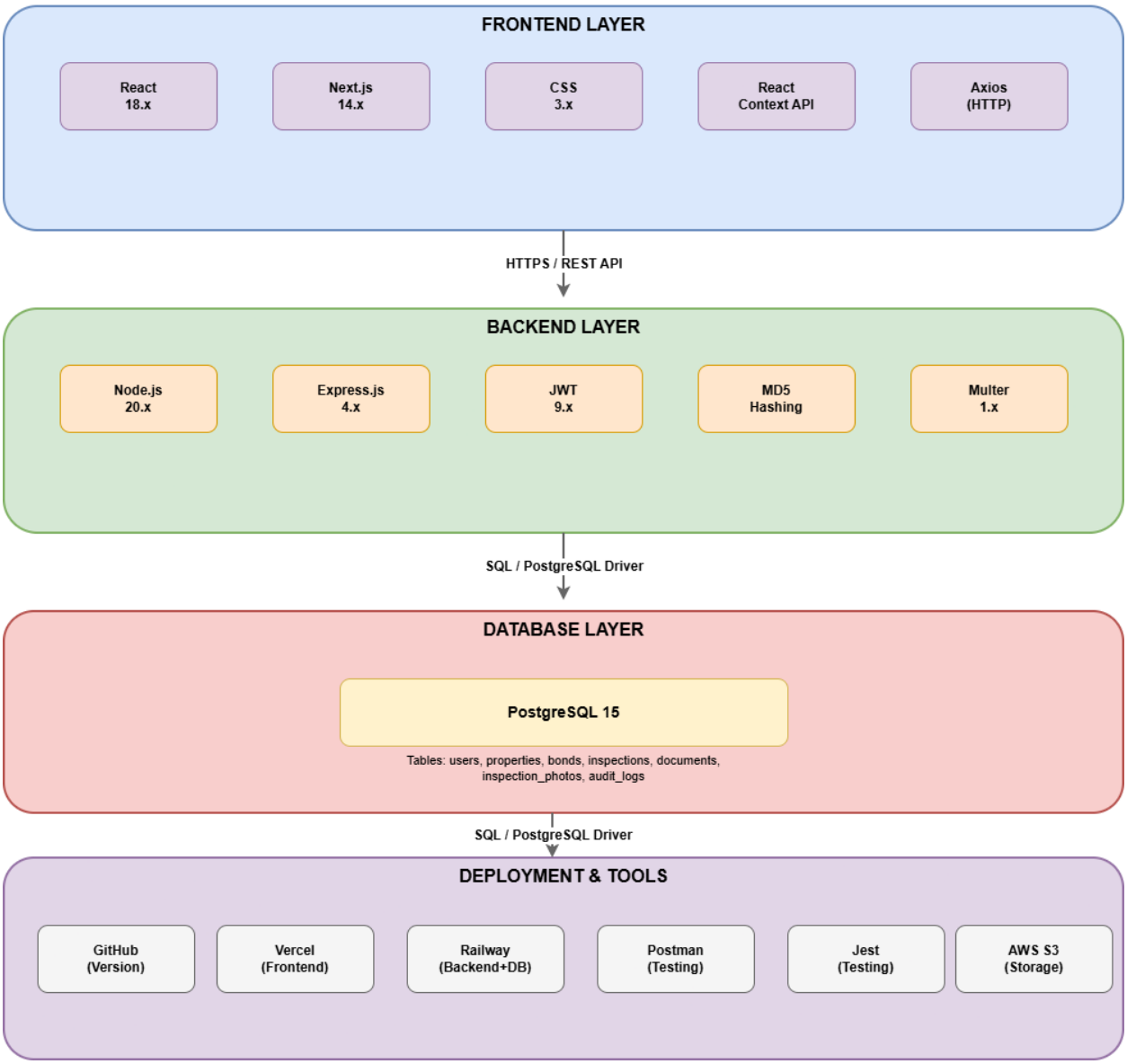
Technology Stack Table

LAYER	TECHNOLOGY	VERSION	PURPOSE	JUSTIFICATION
Frontend	React	18.x	UI Framework	Component-based, fast rendering, large community
Frontend	Next.js	14.x	React Framework	Server-side rendering, routing, SEO friendly
Styling	TailwindCSS	3.x	CSS Framework	Utility-first, responsive design, easy customization
State Management	React Context API	18.x	State Management	Built-in, lightweight, no extra dependencies
Backend	Node.js	20.x	JavaScript Runtime	Fast, scalable, NPM ecosystem
Backend	Express.js	4.x	API Framework	Minimal, flexible, middleware support
Authentication	JWT (JSON Web Token)	9.x	Token-based Auth	Stateless, secure, works with REST APIs

LAYER	TECHNOLOGY	VERSION	PURPOSE	JUSTIFICATION
Password Encryption	MD5	-	Hashing Algorithm	Required by project specification
File Upload	Multer	1.x	Middleware	Handles multipart/form-data, file validation
Database	PostgreSQL	15.x	Relational Database	ACID compliant, reliable, free
API Testing	Postman	Latest	API Testing	Industry standard, easy to use
Unit Testing	Jest	29.x	Testing Framework	React recommended, fast
End-to-End Testing	Selenium	4.x	Automated Testing	Browser automation, cross-browser testing
Version Control	Git	Latest	Source Control	Industry standard
Repository Hosting	GitHub	-	Code Hosting	Free for students, collaboration features
Frontend Hosting	Vercel	-	Deployment	Free tier, auto-deploy from GitHub

LAYER	TECHNOLOGY	VERSION	PURPOSE	JUSTIFICATION
Backend Hosting	Railway	-	Deployment	Free tier, easy PostgreSQL integration
Document Storage	AWS S3	-	File Storage	Free tier (5GB), reliable
Wireframing	Balsamiq Cloud / Figma	-	UI Design	Student license available
Diagramming	draw.io	-	Diagrams (DFD, ERD)	Free, export to XML/PNG
Documentation	Microsoft Word	-	SRS Report	Provided by Kent Institute
Communication	Microsoft Teams / WhatsApp	-	Team Communication	Free, group chat

Technology stack



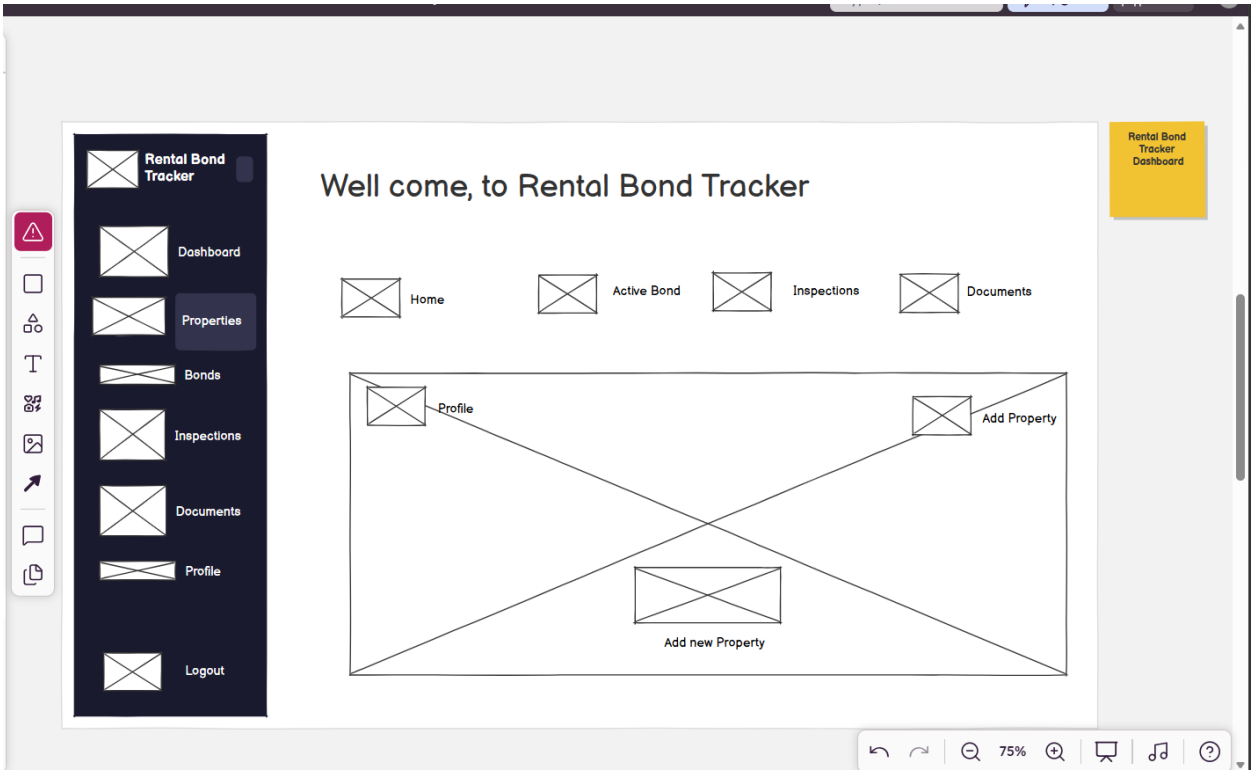
4.1.1 SYSTEM DESIGN – BALSAMIQ CLOUD

The user interface for Rental Bond Tracker is designed using Balsamiq Cloud. The following screens have been created:

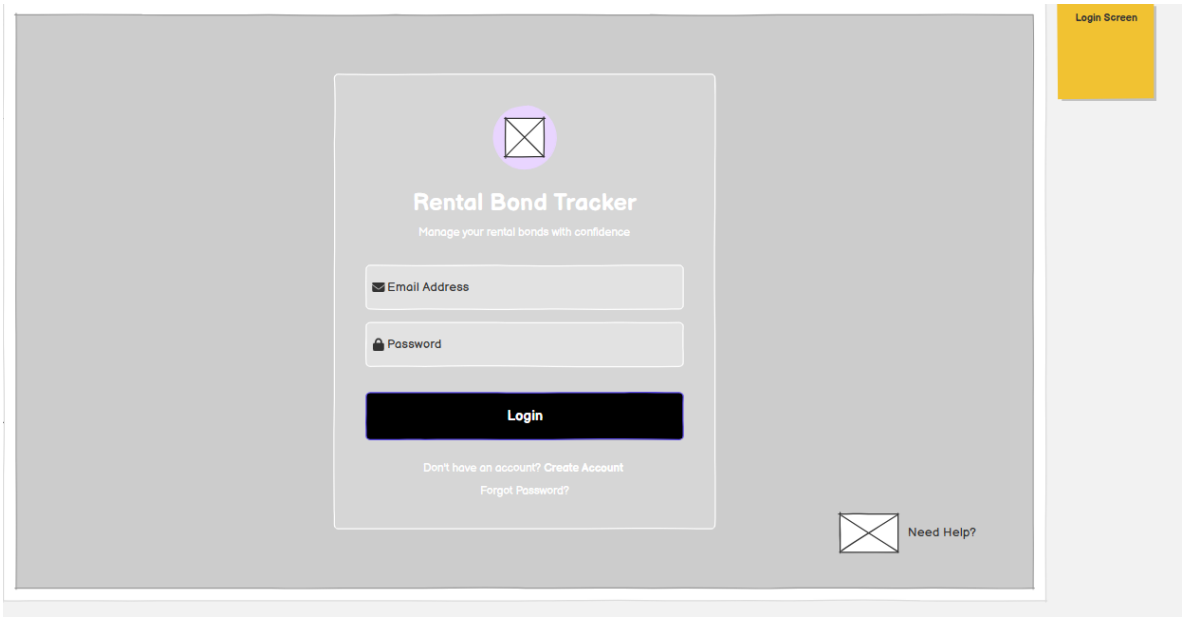
SCREEN	DESCRIPTION	ASSIGNEE
Login Page	Email and password fields, login button, register link	Linh
Register Page	Full name, email, password, phone fields	Linh
Dashboard	Stats cards, recent documents, quick action buttons	subham
Add Property Form	Address, landlord, agent, lease dates	Asim
Edit Property Form	Pre-filled form for editing	Asim
Bond Tracking Form	Amount, date, reference, status	Asim
Bond History Table	List of all bonds with status	Asim
Inspection Form	Date, type, notes, photo upload	Linh
Document Upload	File picker, title, description	subham
User Profile	View and edit profile	Asim
Audit Logs (Admin)	Table of system activities	Asim

Here is the design from balsamiq.cloud

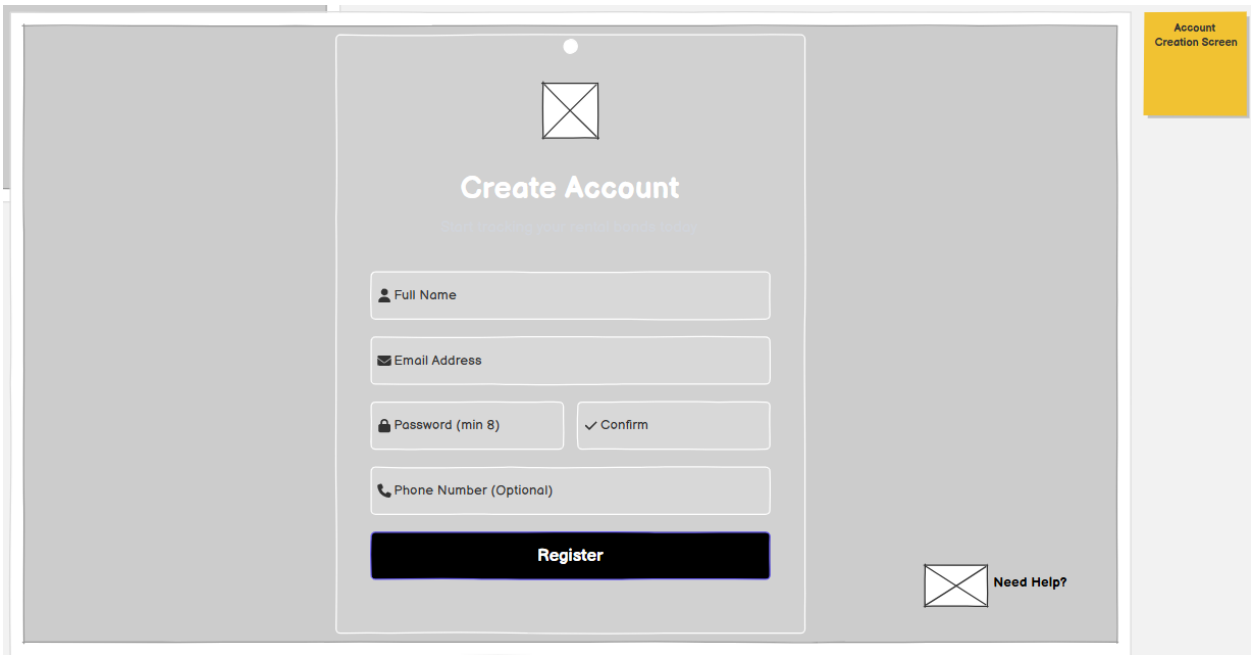
Home page



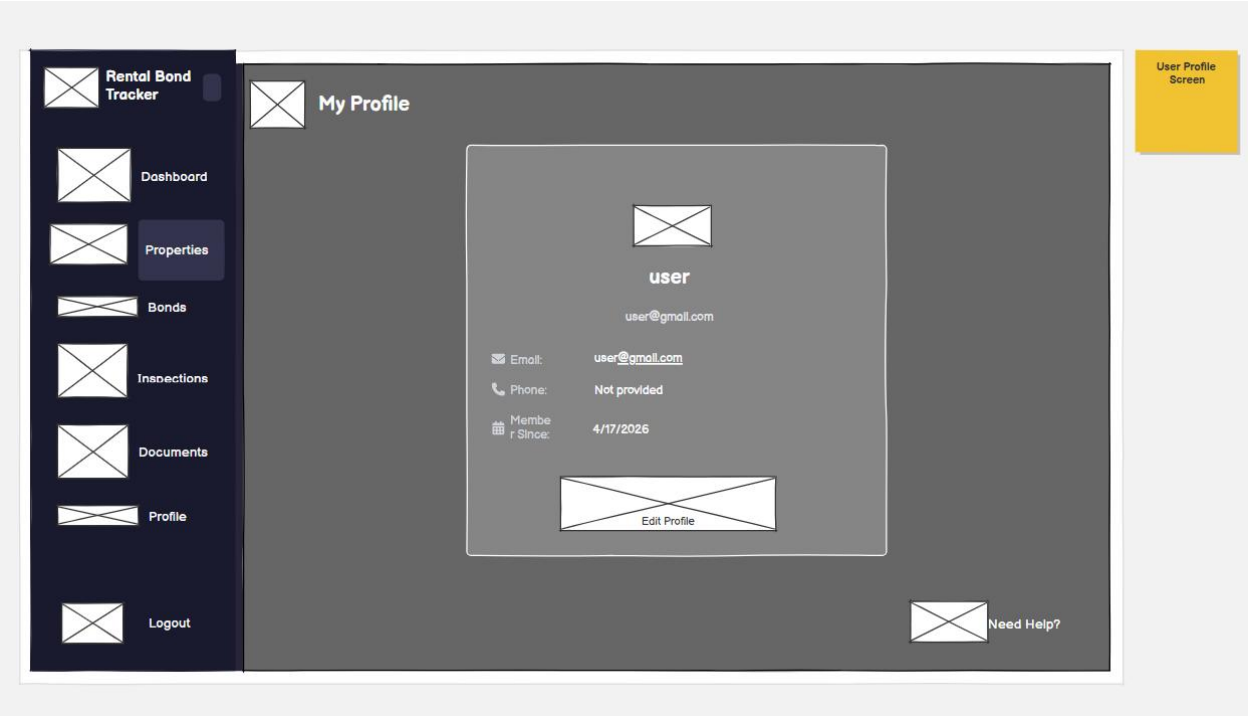
Login system

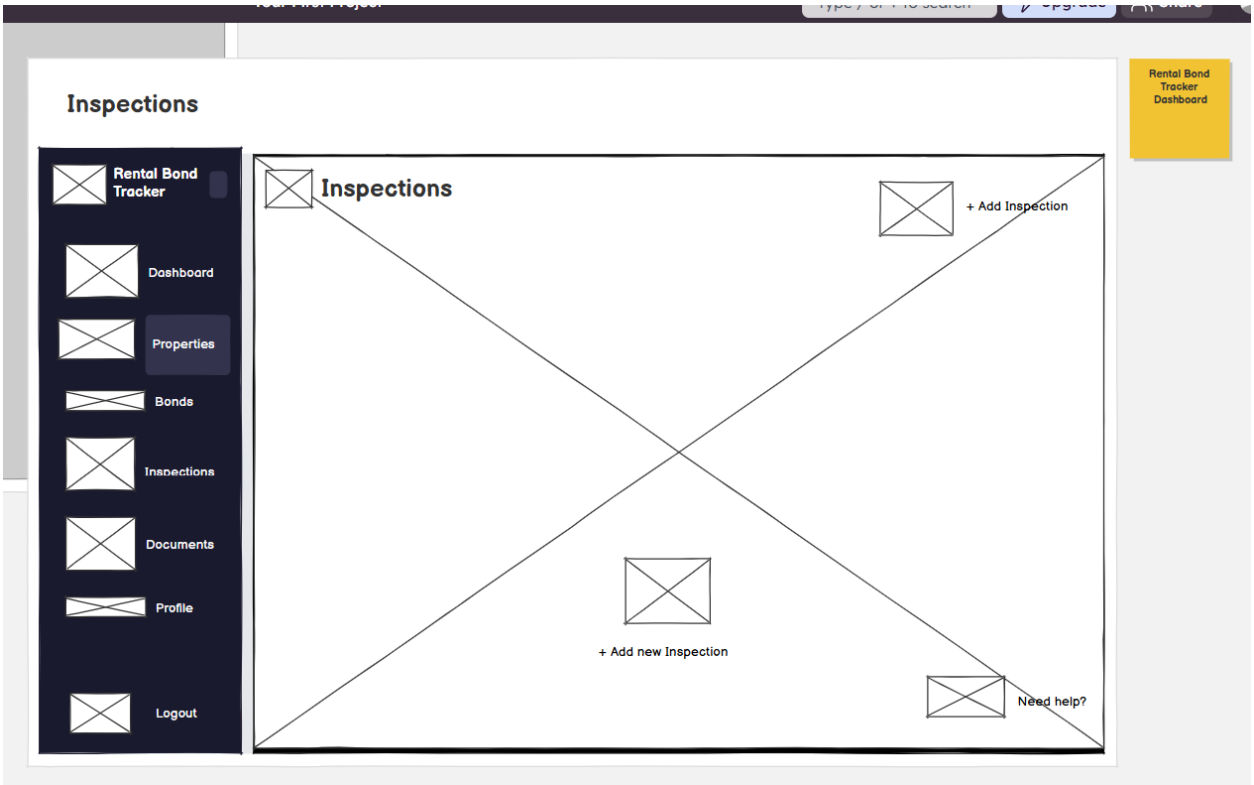


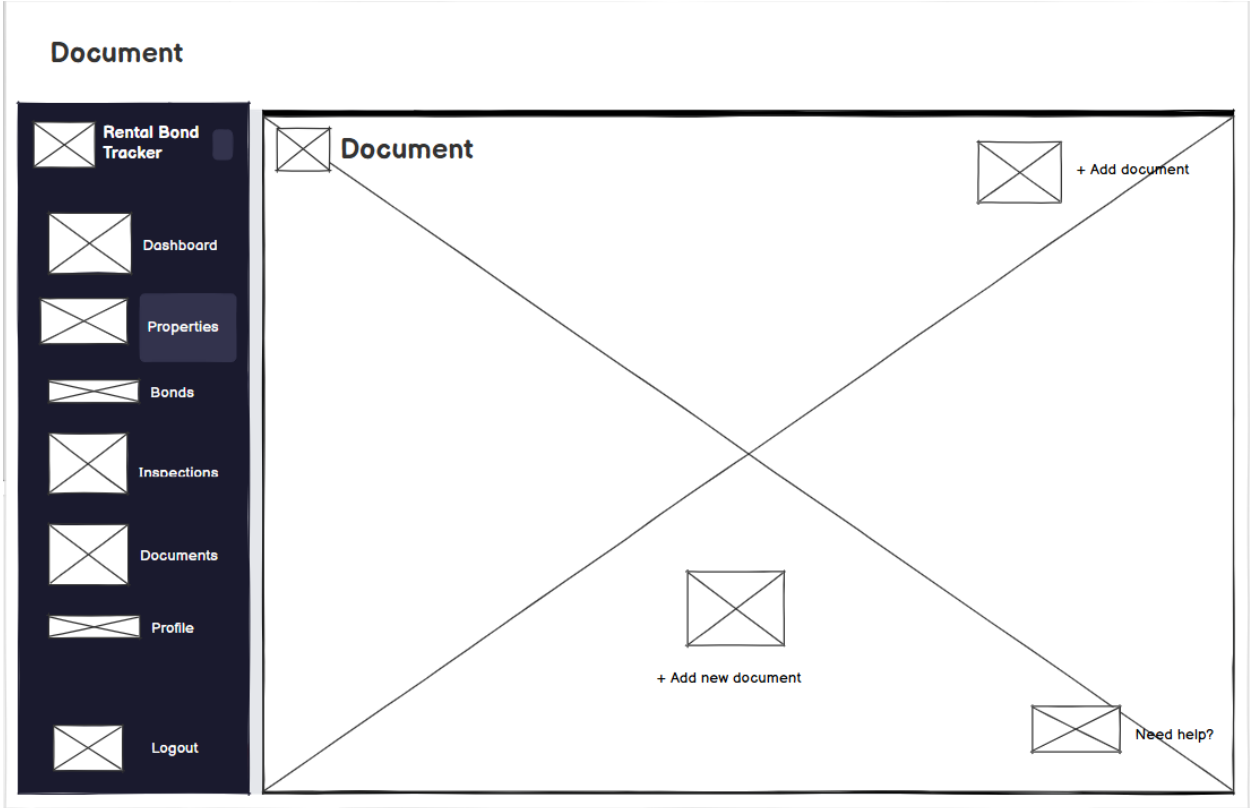
Register

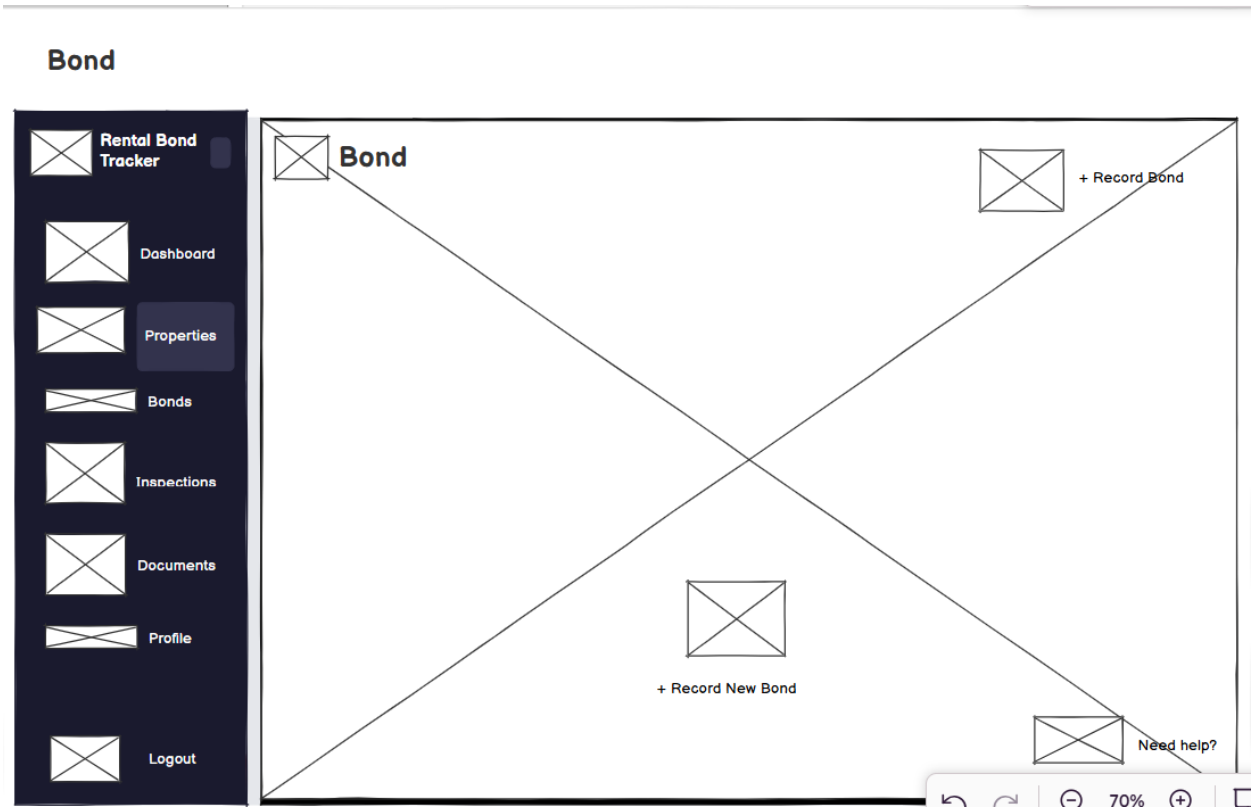


Profile




Inspections
Rental Bond Tracker Dashboard
Rental Bond Tracker
Dashboard
Properties
Bonds
Inspections
Documents
Profile
Logout
Inspections
+ Add Inspection
+ Add new Inspection
Need help?


Document
Rental Bond Tracker
Dashboard
Properties
Bonds
Inspections
Documents
Profile
Logout
Document
+ Add document
+ Add new document
Need help?



4.1.2 UI WIREFRAMES

The User Interface (UI) for Rental Bond Tracker follows a modern glassmorphism design with a full-screen video background. The design philosophy focuses on simplicity, clarity, and ease of use for tenants managing their rental bonds.

Design Principles

PRINCIPLE	DESCRIPTION
Glassmorphism	Transparent frosted glass effect with backdrop blur creates a modern, premium feel
Consistent Layout	Fixed sidebar navigation with collapsible menu for easy access to all features

PRINCIPLE	DESCRIPTION
Responsive Design	Adapts to desktop, tablet, and mobile screen sizes
Visual Hierarchy	Clear headings, card-based layouts, and color-coded status indicators
User-Friendly Forms	Simple, well-spaced input fields with icons and validation

Key Screens

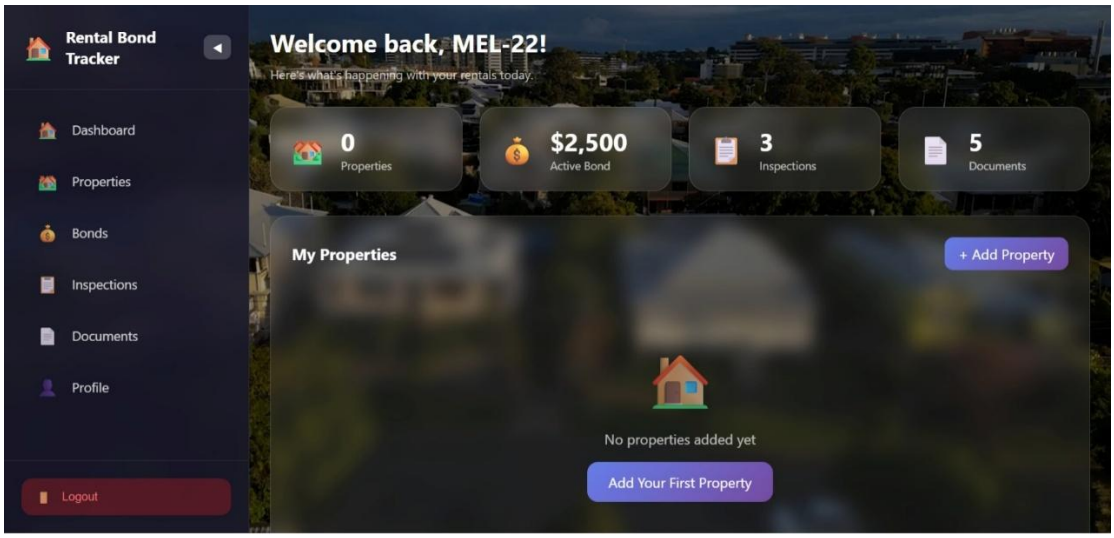
SCREEN	PURPOSE	KEY ELEMENTS
Login	User authentication	Email, password fields, login button, register link
Register	New account creation	Full name, email, password, confirm password, phone
Dashboard	Overview of all activities	Stats cards, recent properties, quick action buttons
Properties	Manage rental properties	Property cards with address, landlord, edit/delete buttons
Bonds	Track bond payments	Bond amount, status badge, payment date, reference number
Inspections	Record property inspections	Inspection type, date, condition notes

SCREEN	PURPOSE	KEY ELEMENTS
Documents	Store tenancy documents	File list with upload, download, delete options
Profile	User account settings	User info, edit profile, change password

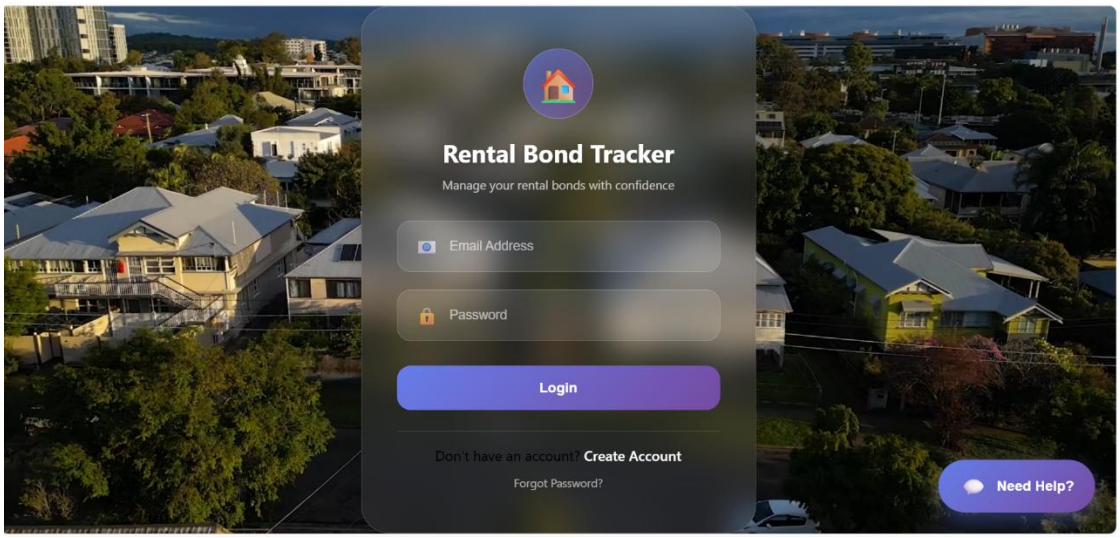
Color Scheme

ELEMENT	COLOR	PURPOSE
Primary Gradient	#667eea to #764ba2	Buttons, logos, highlights
Glass Background	rgba(255,255,255,0.08)	Cards and containers
Text Primary	White	Headings and labels
Text Secondary	rgba(255,255,255,0.7)	Subtitles and descriptions
Success (Paid)	#10b981	Bond status badge
Warning (Pending)	#f59e0b	Bond status badge
Error (Refunded)	#ef4444	Bond status badge

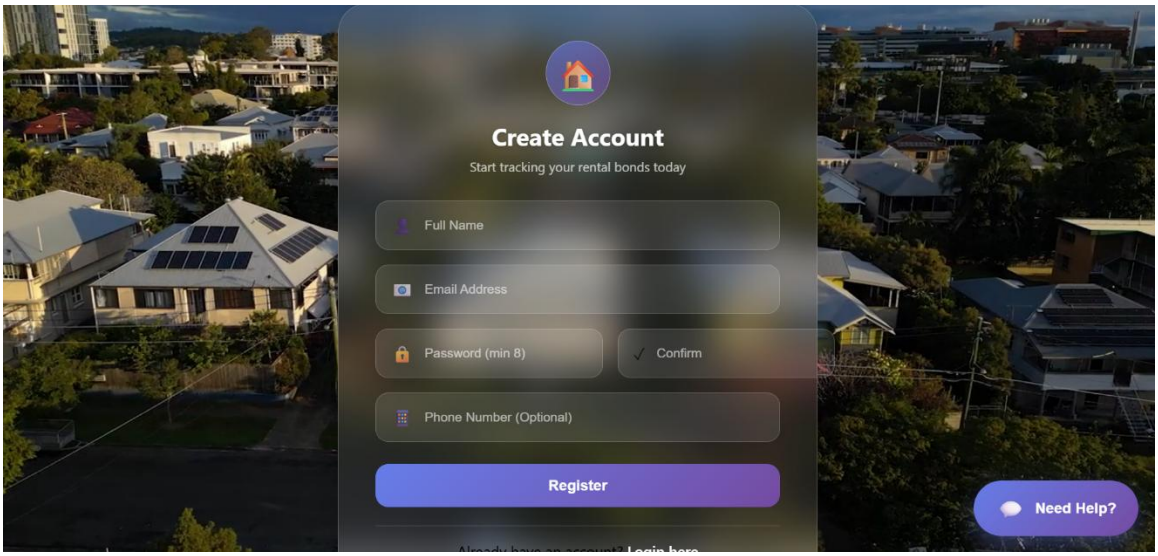
Dashboard



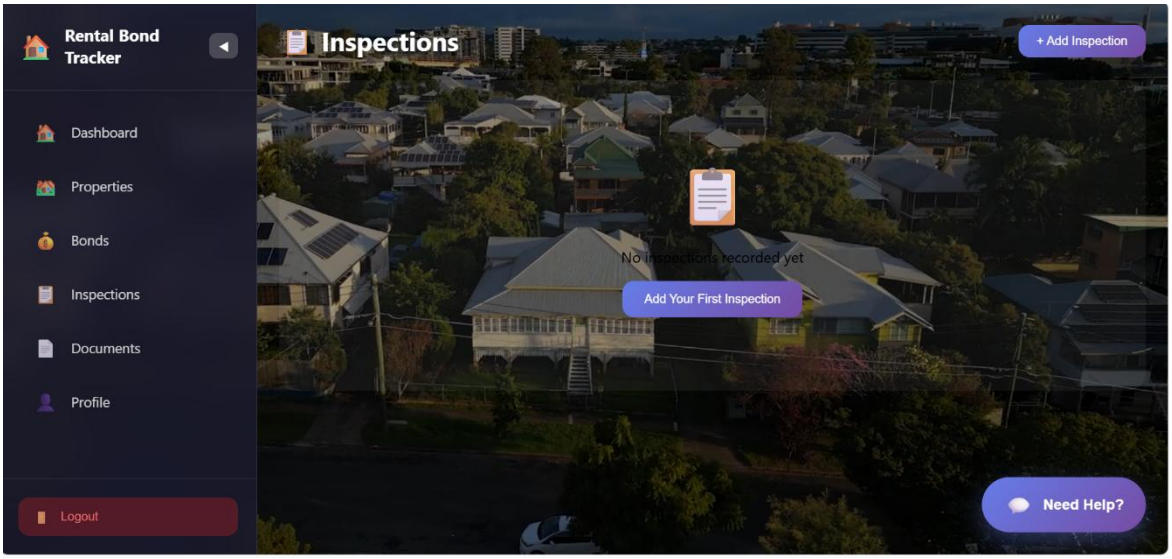
Login



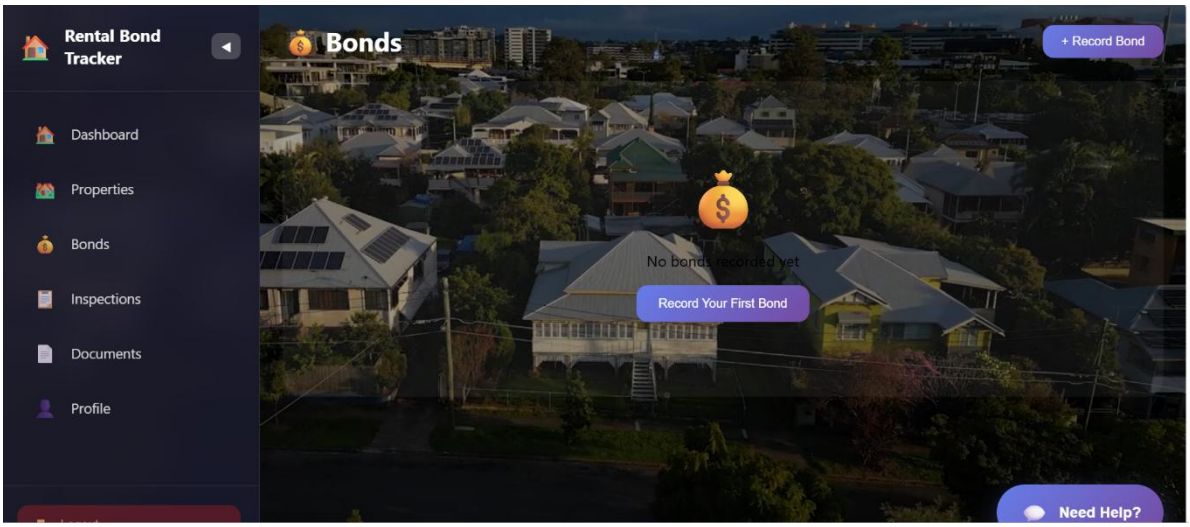
Register



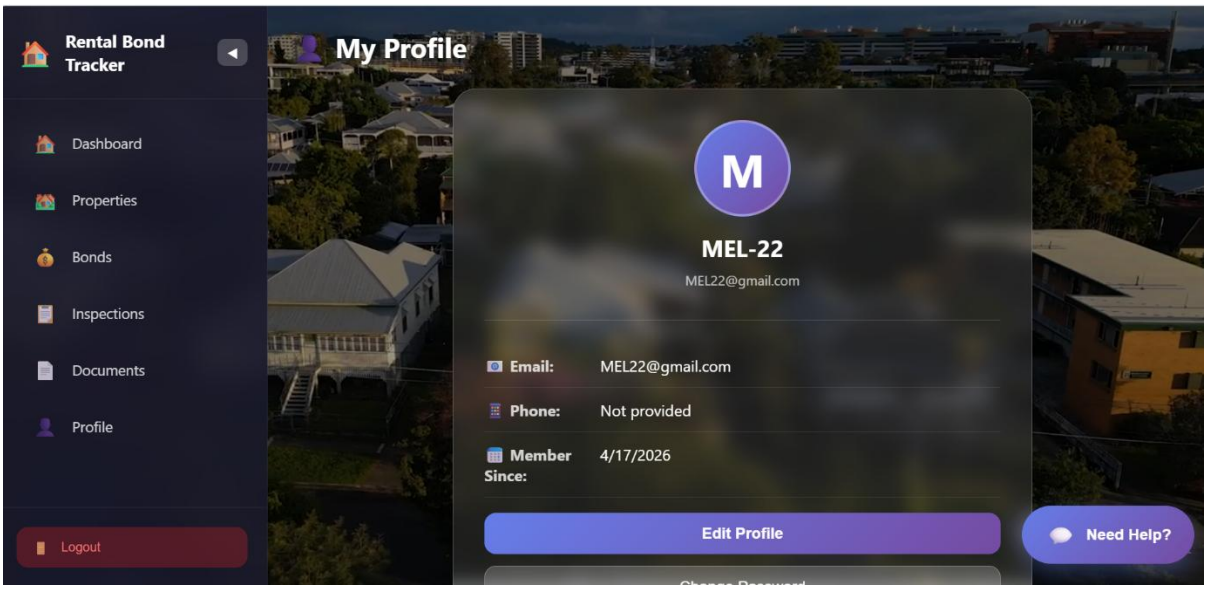
Inspections



Bonds



Profile



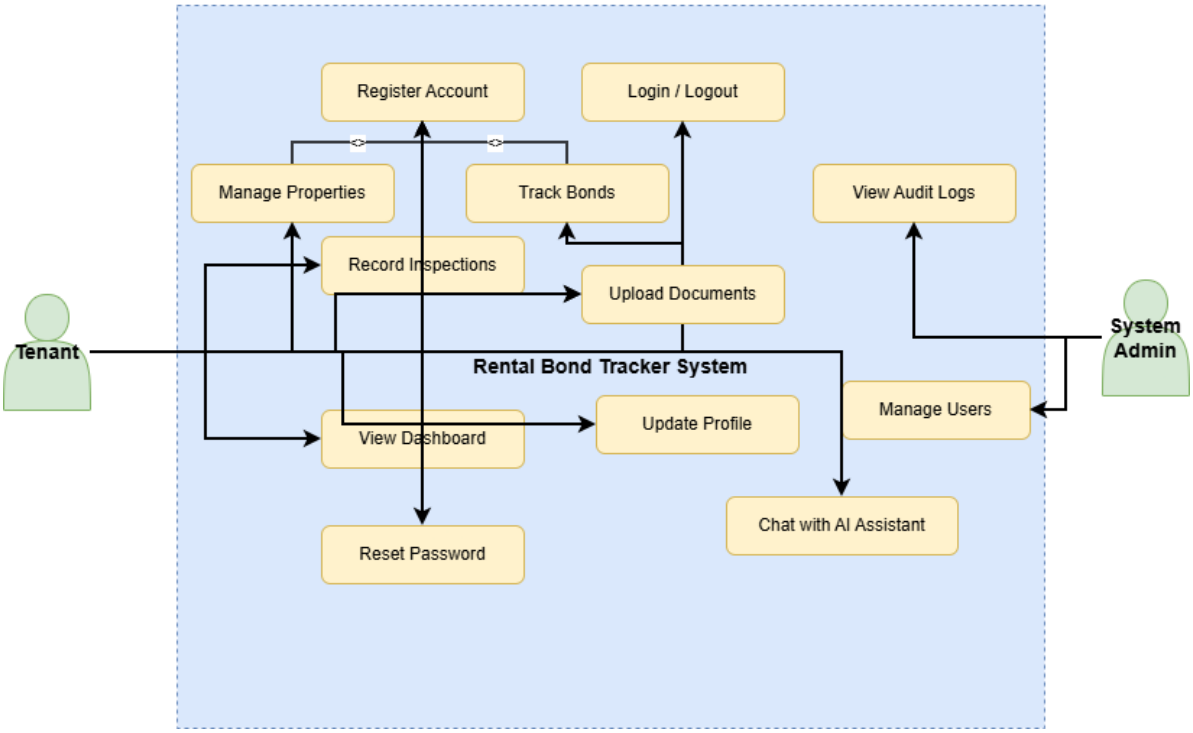
5.1.1 USE CASE DIAGRAM

Actors: Tenant, System Administrator

Use Cases:

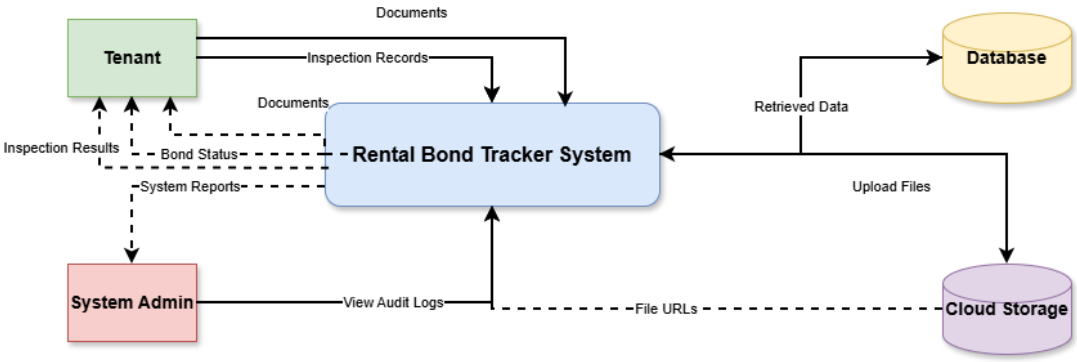
USE CASE	ACTOR	DESCRIPTION
Register Account	Tenant	Create new user account
Login/Logout	Tenant	Authenticate and manage session
Manage Property	Tenant	Add, edit, delete property details
Manage Bond	Tenant	Record bond, update status, view history
Manage Inspection	Tenant	Create inspection, upload photos
Manage Documents	Tenant	Upload, download, delete documents
View Dashboard	Tenant	View bond summary and updates
Update Profile	Tenant	Change profile information
View Audit Logs	Admin	Monitor user activities

Use Case Diagram - Rental Bond Tracker



6.1.1 CONTEXT DIAGRAM

Context Diagram (DFD Level 0) - Rental Bond Tracker



External Entities

ENTITY	DESCRIPTION	ROLE
Tenant	End user	Provides login, property, bond, inspection, document data. Receives dashboard, bond status, inspection results
Database	Data storage	Stores and retrieves all system data
System Admin	Administrator	Views audit logs, manages system
Cloud Storage	File storage	Stores uploaded documents and photos

Data Flows (Input to System)

FLOW	FROM	DESCRIPTION
Login Credentials	Tenant	Email and password for authentication
Property Details	Tenant	Address, landlord, agent details
Bond Details	Tenant	Amount, payment date, status
Inspection Records	Tenant	Date, type, notes, photos
Documents	Tenant	Lease agreements, receipts
View Audit Logs	Admin	Request for system logs

Data Flows (Output from System)

FLOW	TO	DESCRIPTION
Dashboard	Tenant	Statistics and summary
Bond Status	Tenant	Paid/Pending/Refunded status
Inspection Results	Tenant	Condition reports
Documents	Tenant	Uploaded files for download
System Reports	Admin	Audit logs and analytics
Store Data	Database	Save records
Query Data	Database	Request records
Retrieved Data	Database	Return records
Upload Files	Cloud Storage	Save documents
File URLs	Cloud Storage	Access links

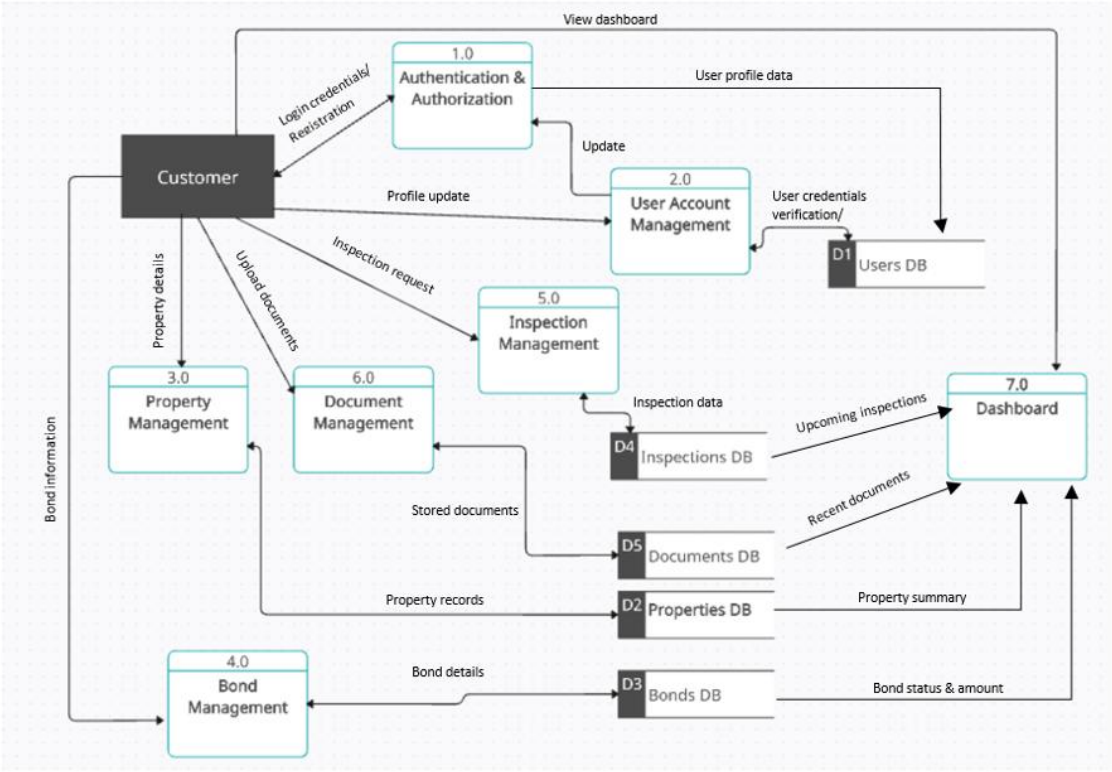
Legend

LINE TYPE	MEANING
Solid Arrow (→)	Input data flow (into system)

LINE TYPE	MEANING
Dashed Arrow (⇢)	Output data flow (from system)
Rectangle	External entity
Rounded Rectangle	System boundary
Cylinder	Data store

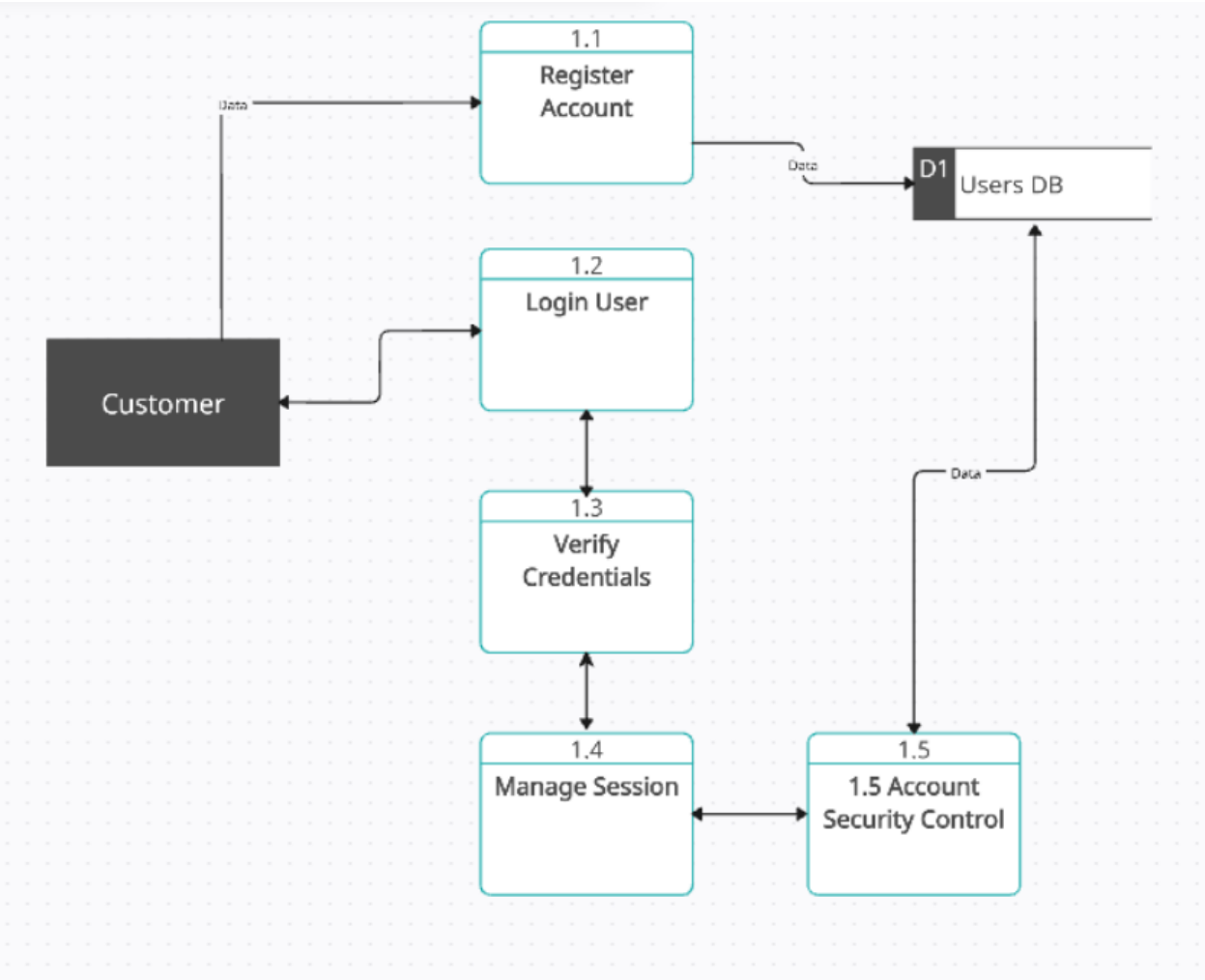
7.1.1 DFD LEVEL 1

DFD level 1 Rental Bond Tracker

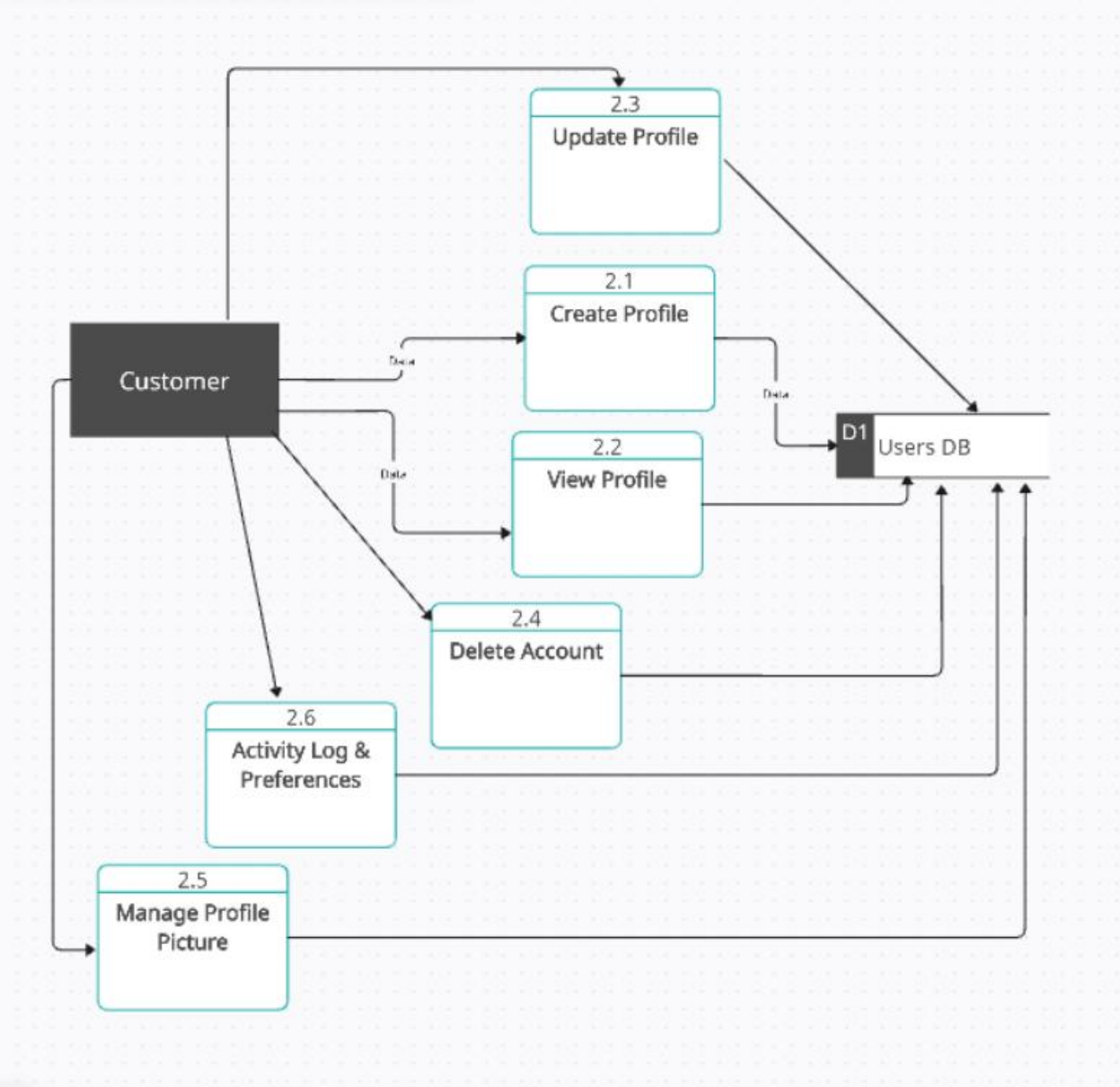


7.1.2 DFD LEVEL 2 RENTAL BOND TRACKER

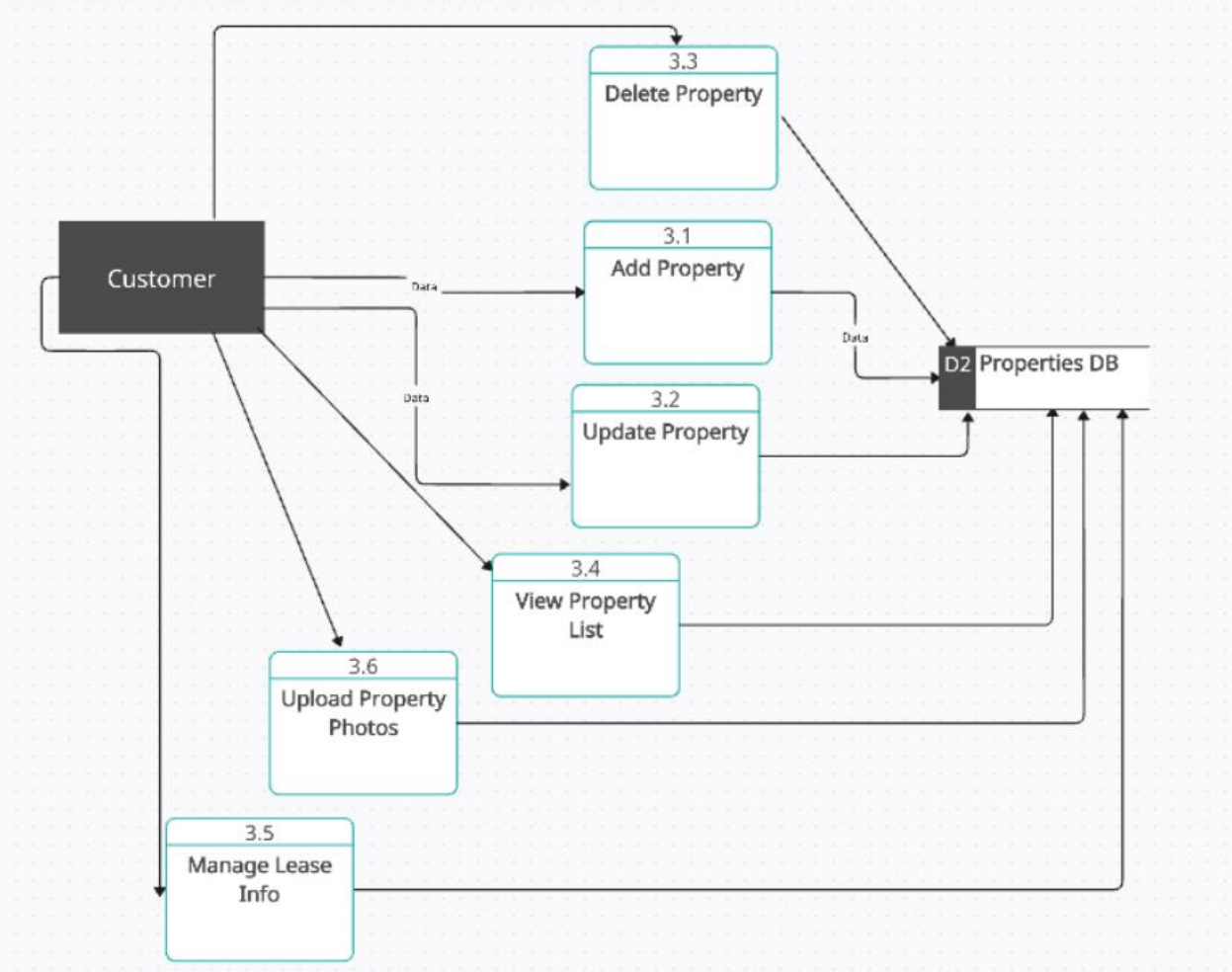
User Authentication



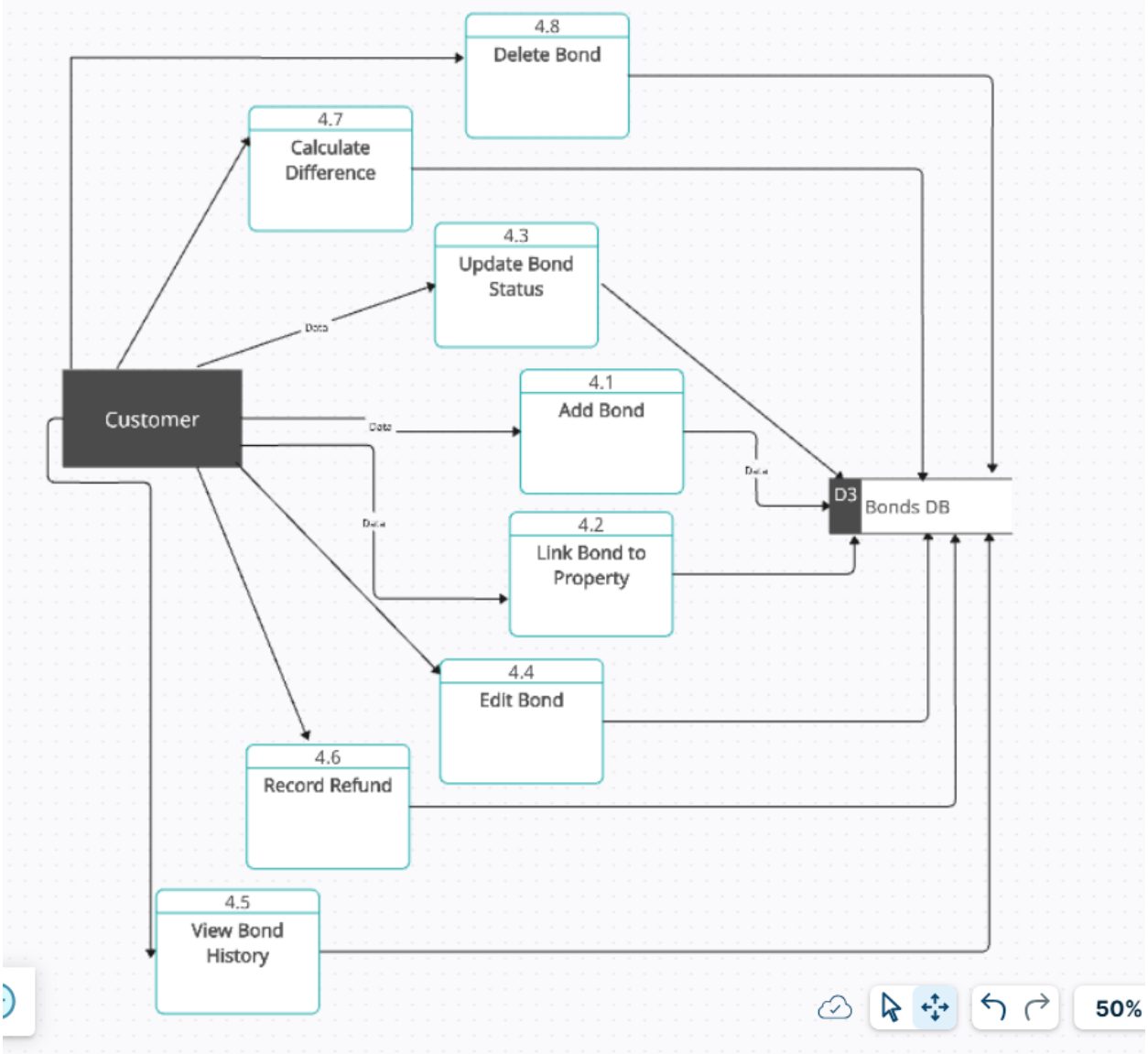
Profile management



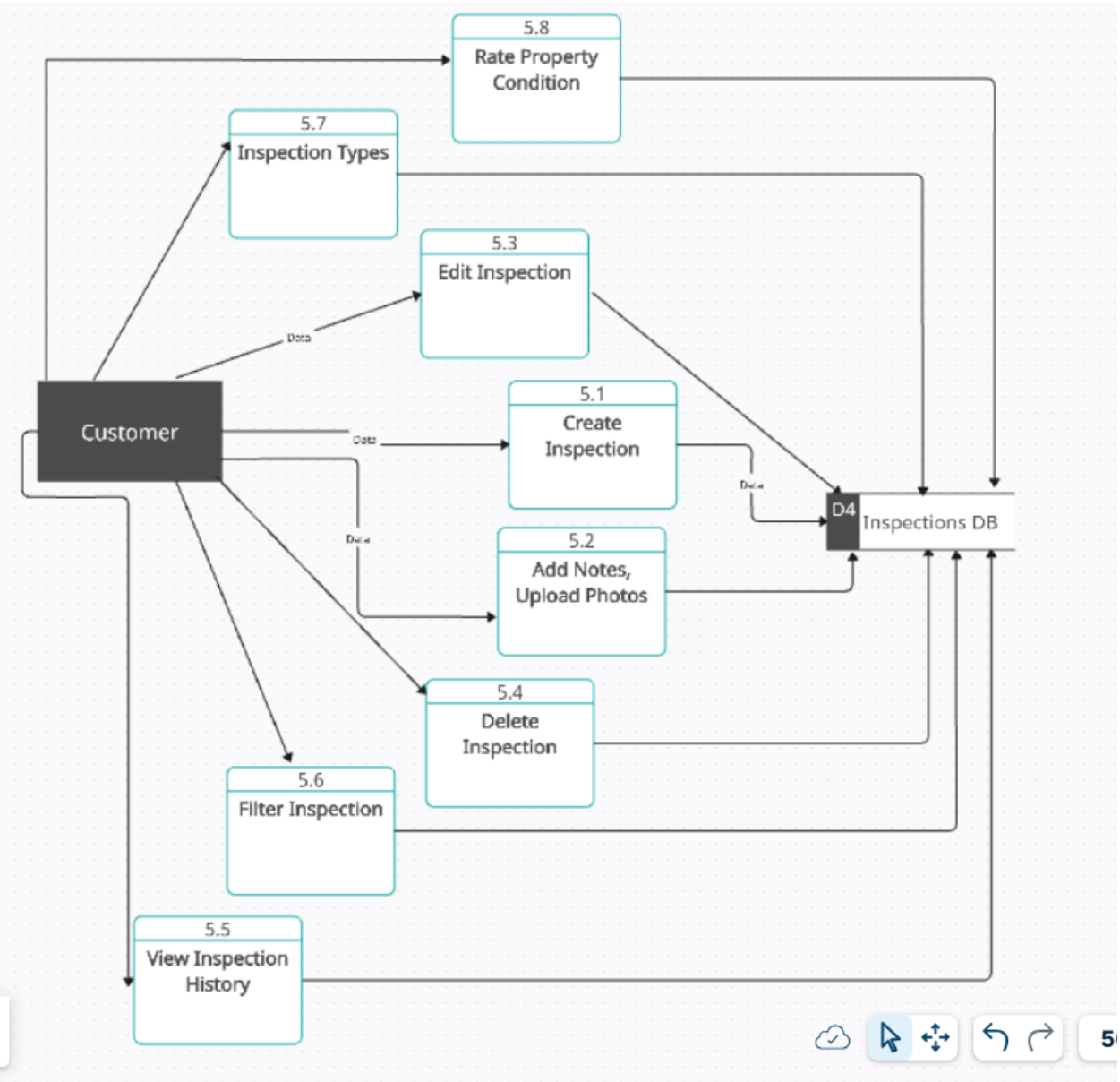
Property Management



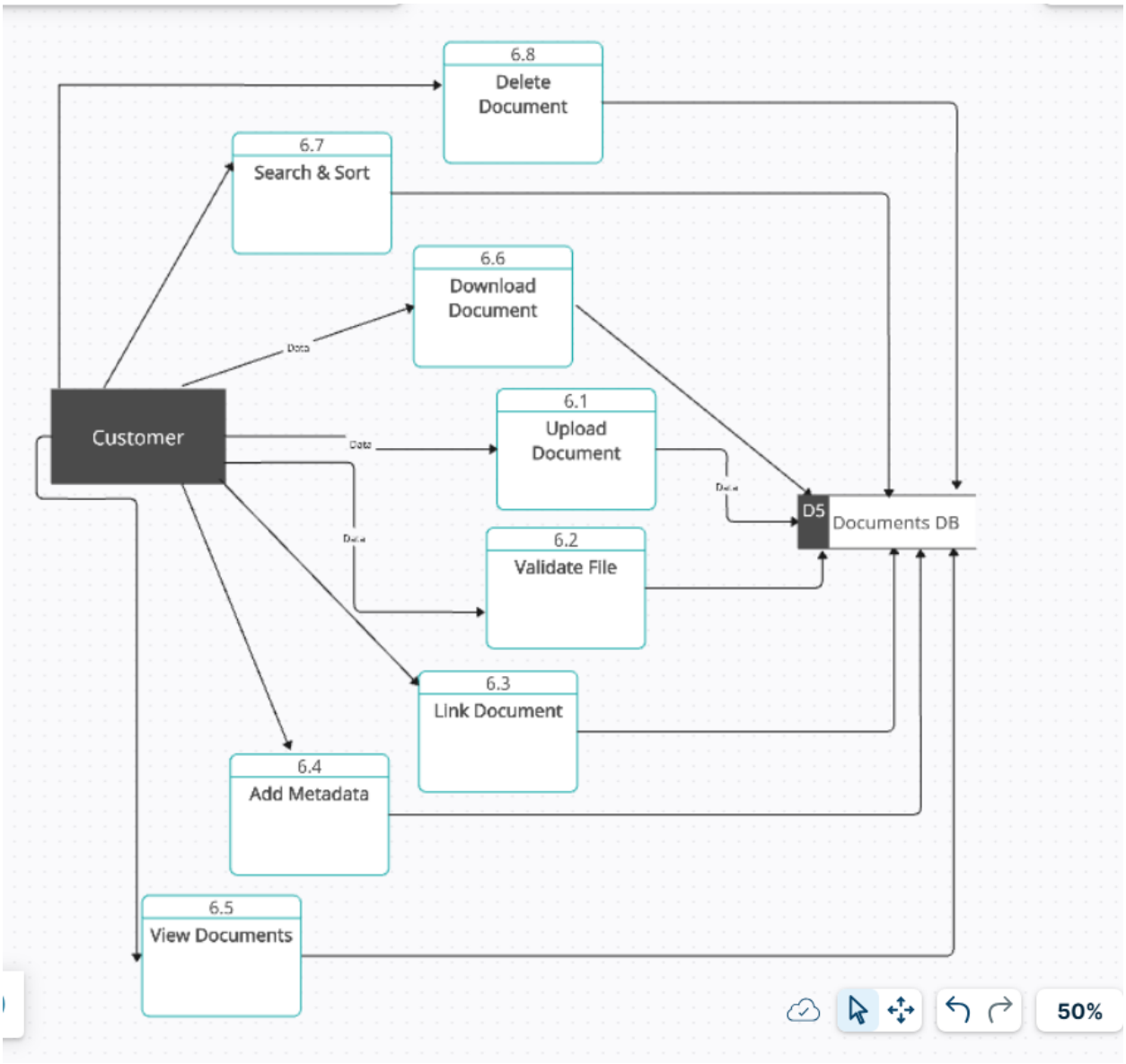
Bond Management



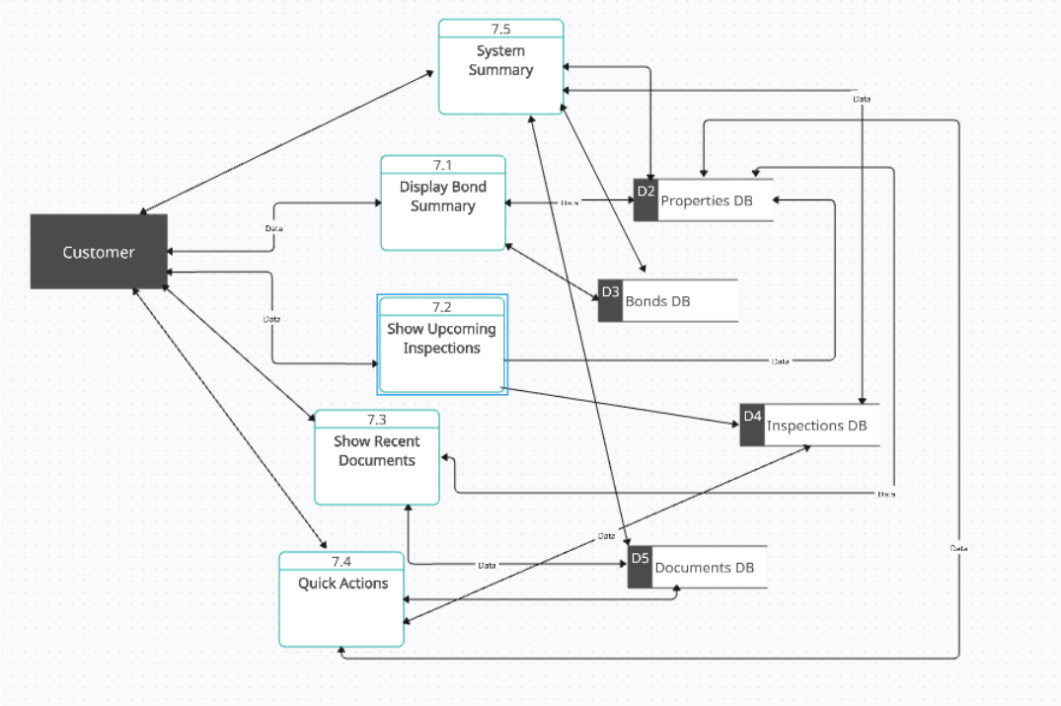
Inspection management



Document management

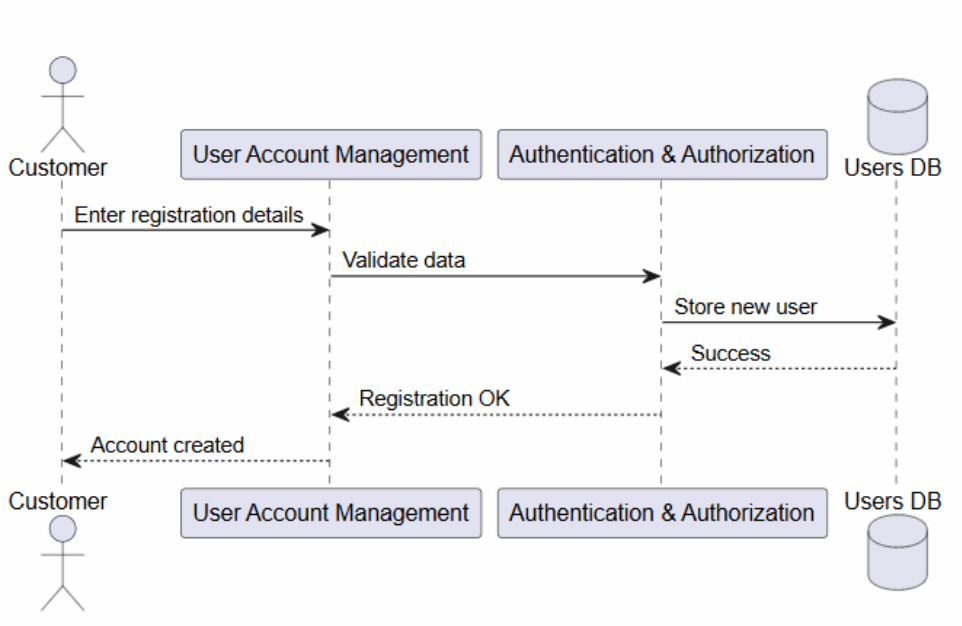


Dashboard



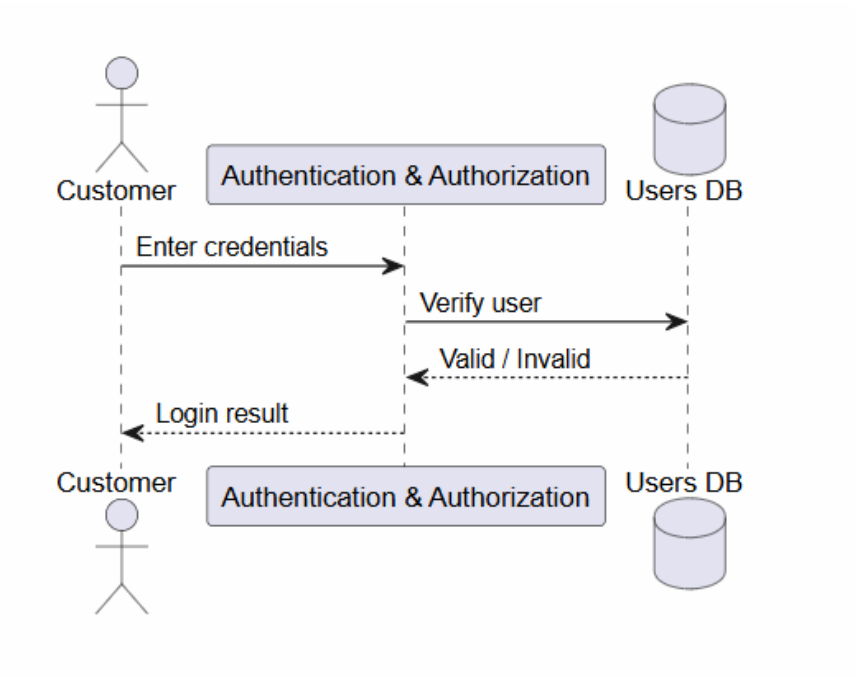
8.1.1 SEQUENCE DIAGRAM

1. Register



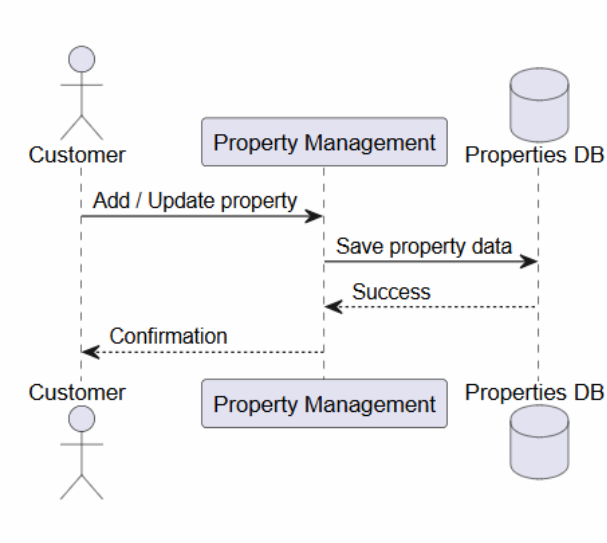
This Sequence Diagram illustrates the process of a user creating a new account. The customer enters registration information, the system verifies its validity, and saves the data to the user database. After successful registration, the account is created and ready for use.

2. login



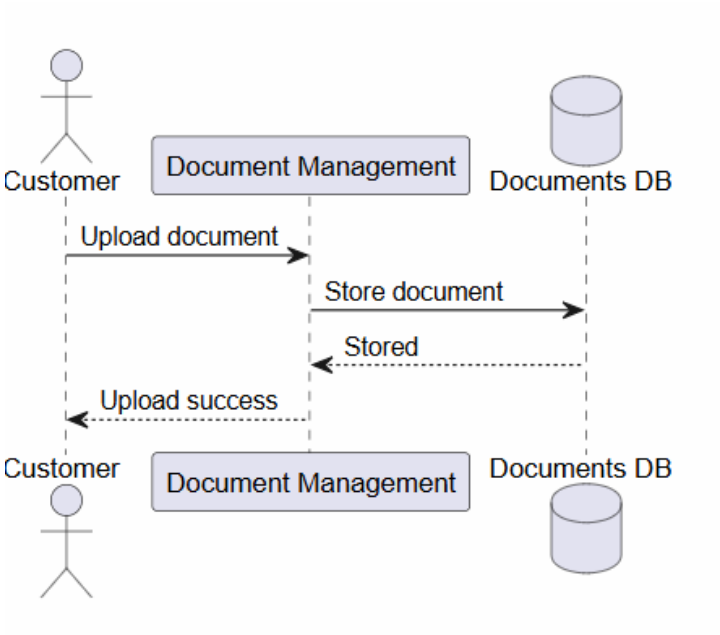
This diagram illustrates the user login process. The user enters their authentication information, and the system checks it against the database. If the information is valid, the user is successfully logged in; otherwise, an error message is displayed.

3. Manage Property



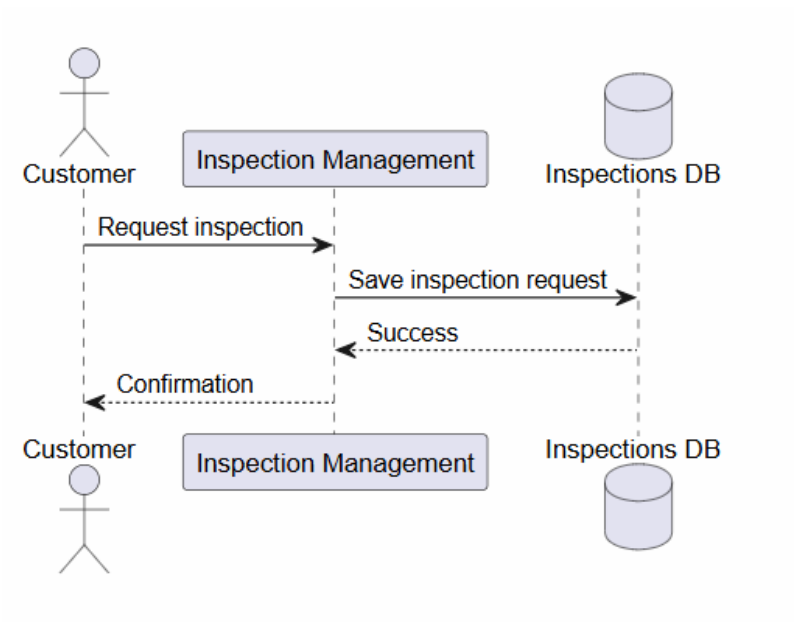
This diagram describes the process of a user adding or updating property information. The data is sent to the management module and stored in the database. The system then returns a confirmation to the user.

4. Upload Document



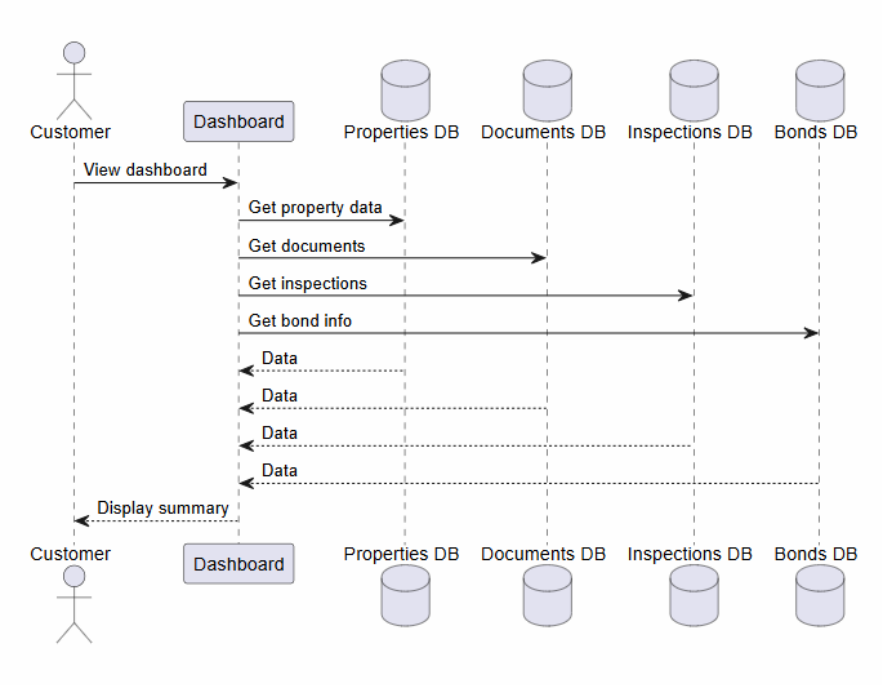
This sequence diagram describes the process of uploading documents to the system. The user submits the file, the system processes and stores it in the document database. Upon completion, the system notifies the user of a successful upload.

5. Request Inspection



This diagram illustrates the process of a user requesting an inspection. The request is sent to the system, stored in the database, and confirmed by the user after successful completion.

6. View dashboard



This Sequence Diagram describes how the system aggregates data to display the dashboard. The system retrieves information from multiple sources (property, document, inspection, bond) and returns an overview interface to the user.

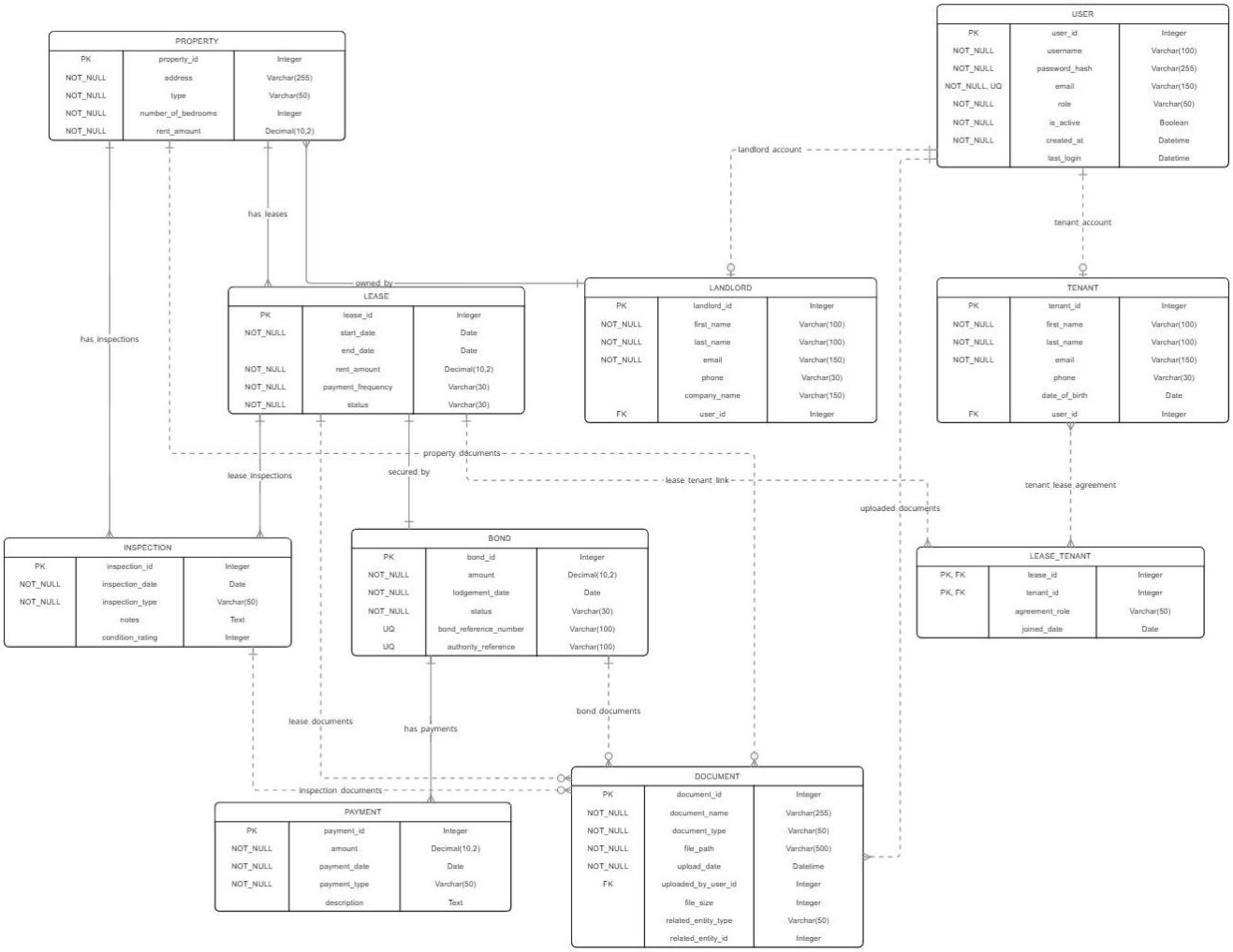
9.1.1 Entity Relationship Diagram (ERD)

TABLE	PURPOSE
users	Tenant account information
properties	Rental property details

TABLE	PURPOSE
bonds	Bond payment records
inspections	Inspection reports
documents	Uploaded files
audit_logs	System activity logs

Relationships

RELATIONSHIP	TYPE
users → properties	One to Many
properties → bonds	One to Many
properties → inspections	One to Many
inspections → photos	One to Many
users → documents	One to Many
users → audit_logs	One to Many



9.1.2 DATA DICTIONARY

Users table:

Column	Data Type	Constraints	Description
user_id	INTEGER	PRIMARY KEY, AUTO_INCREMENT	Unique user identifier

Column	Data Type	Constraints	Description
full_name	VARCHAR(100)	NOT NULL	User's full name
email	VARCHAR(100)	UNIQUE, NOT NULL	Login email
password	VARCHAR(255)	NOT NULL	MD5 encrypted password
phone	VARCHAR(20)	NULL	Contact number
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Account creation date
updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Last update
last_login	TIMESTAMP	NULL	Last login timestamp

Properties table:

Column	Data Type	Constraints	Description
property_id	INTEGER	PRIMARY KEY, AUTO_INCREMENT	Unique property identifier
user_id	INTEGER	FOREIGN KEY (users)	Owner tenant ID
address	TEXT	NOT NULL	Property address

Column	Data Type	Constraints	Description
landlord_name	VARCHAR(100)	NOT NULL	Landlord full name
landlord_phone	VARCHAR(20)	NULL	Landlord contact
agent_name	VARCHAR(100)	NULL	Property agent name
lease_start	DATE	NULL	Lease start date
lease_end	DATE	NULL	Lease end date
is_current	BOOLEAN	DEFAULT FALSE	Current residence flag

Bonds table:

Column	Data Type	Constraints	Description
bond_id	INTEGER	PRIMARY KEY, AUTO_INCREMENT	Unique bond identifier
property_id	INTEGER	FOREIGN KEY (properties)	Associated property
amount	DECIMAL(10,2)	NOT NULL	Bond amount in AUD
payment_date	DATE	NOT NULL	Date bond paid
status	VARCHAR(20)	DEFAULT 'Pending'	Paid/Pending/Refunded

Column	Data Type	Constraints	Description
refund_amount	DECIMAL(10,2)	NULL	Amount refunded

Inspections table:

Column	Data Type	Constraints	Description
inspection_id	INTEGER	PRIMARY KEY, AUTO_INCREMENT	Unique inspection identifier
property_id	INTEGER	FOREIGN KEY (properties)	Associated property
inspection_date	DATE	NOT NULL	Date of inspection
inspection_type	VARCHAR(20)	NOT NULL	Entry/Exit/Routine
condition_notes	TEXT	NULL	Detailed condition notes

Documents table:

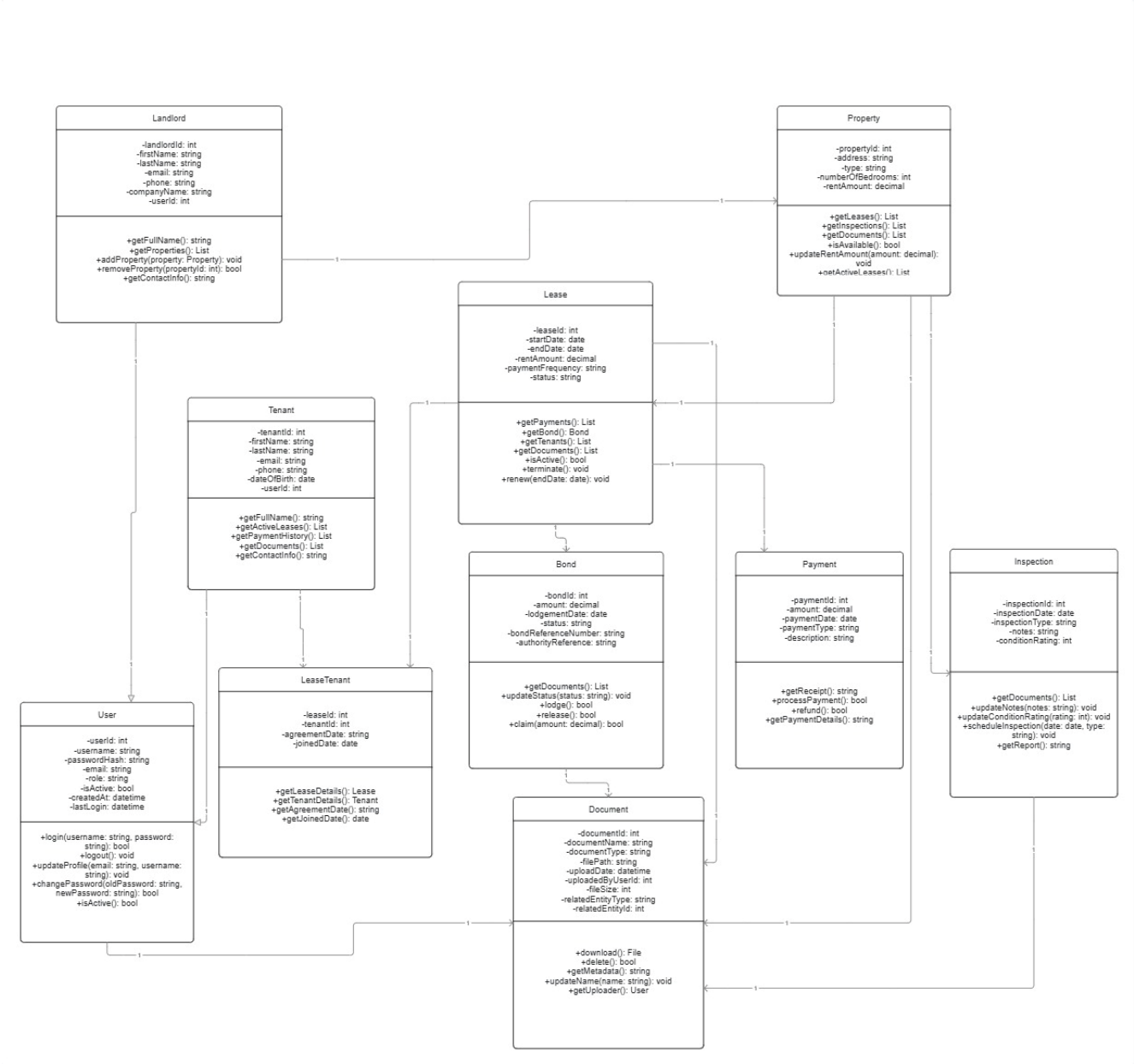
Column	Data Type	Constraints	Description
document_id	INTEGER	PRIMARY KEY, AUTO_INCREMENT	Unique document identifier

Column	Data Type	Constraints	Description
user_id	INTEGER	FOREIGN KEY (users)	Uploading user
title	VARCHAR(200)	NOT NULL	Document title
file_name	VARCHAR(255)	NOT NULL	Original file name
file_path	VARCHAR(255)	NOT NULL	Stored file path
file_type	VARCHAR(50)	NOT NULL	MIME type

Audit logs table:

Column	Data Type	Constraints	Description
log_id	INTEGER	PRIMARY KEY, AUTO_INCREMENT	Unique log identifier
user_id	INTEGER	FOREIGN KEY (users)	User who performed action
action_type	VARCHAR(20)	NOT NULL	CREATE/READ/UPDATE/DELETE/LOGIN
description	TEXT	NOT NULL	Action description
timestamp	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Action timestamp

10.1.1 CLASS DIAGRAM



11.1.1 DATA PROTECTION & IMPACT ASSESSMENT

A Data Protection Impact Assessment identifies privacy risks and how we protect tenant data.

Step 1: Identify the Need for a DPIA

The Rental Bond Tracker is a web-based application that collects, stores, and processes tenant personal data including names, emails, phone numbers, property addresses, bond amounts, and uploaded documents.

Why DPIA is required:

Reasons	Description
Personal data collection	Tenants register with name, email, phone
Financial data	Bond amounts, payment dates, refund records
Document storage	Lease agreements, receipts, inspection photos
Data sharing	Documents stored on AWS S3 cloud
Audit logging	All user actions are recorded

Step 2: Describe the Processing

Nature of Processing:

Action	How it works
Collection	Tenant enters data via registration, property, bond, inspection forms
Use	Data displayed on dashboard, used for bond tracking

Action	How it works
Storage	PostgreSQL database, AWS S3 for documents
Deletion	Account deletion removes all associated data

Data Collected:

Data Type	Examples
Personal	Name, email, phone number
Property	Address, landlord name, agent name
Financial	Bond amount, payment date, refund amount
Documents	Lease agreements, receipts, inspection photos

Data Sharing with Third Parties:

Third Party	Data Shared	Purpose
AWS S3	Documents, photos	File storage
Vercel	Frontend data	Web hosting
Railway	Backend data	API and database hosting

Data Retention:

Data Type	Retention Period
User account	Until user deletes account
Bond records	Until account deletion
Audit logs	12 months
Documents	Until user deletes them

Step 3: Consultation Process

Stakeholder	Consultation Method	Outcome
Tenants	Requirements gathering	Need for bond tracking confirmed
Project Supervisor	Weekly meetings	Technical guidance provided
Development Team	Daily standups	Privacy controls implemented

Step 4: Assess Necessity and Proportionality

Lawful Basis for Processing (Australian Privacy Principles):

Purpose	Lawful Basis
Account registration	Tenant consent

Purpose	Lawful Basis
Bond tracking	Contractual necessity
Document storage	Tenant consent
Audit logging	Legitimate interest

Data Minimisation:

Measure	Implementation
Only necessary data collected	Name, email, bond amount only
Optional fields	Phone number is optional
No sensitive data	Health, bank details not collected

Information Provided to Individuals:

Document	Location
Privacy Notice	Registration page
Terms of Service	Footer of website

Step 5: Identify and Assess Risks

Risk Description	Likelihood	Severity	Overall Risk
Unauthorised access to tenant data	Possible	Significant	**Medium**
Password theft from database	Remote	Severe	**Medium**
Data interception during transmission	Possible	Significant	**Medium**
SQL injection attack	Possible	Significant	**Medium**
Data breach from cloud storage	Remote	Severe	**Low**
Audit logs retained indefinitely	Probable	Significant	**High**
No privacy notice displayed	Probable	Significant	**High**

Step 6: Identify Measures to Reduce Risk

Risk	Mitigation Measure	Residual Risk
Unauthorised access	Login required, session timeout (30 min), RBAC	**Low**
Password theft	MD5 encryption (non-reversible)	**Low**
Data interception	HTTPS encryption for all data	**Low**
SQL injection	Parameterized queries	**Low**

Risk	Mitigation Measure	Residual Risk
Data breach	Users only access their own data	**Low**
Audit logs retained	Define retention policy (12 months), auto-delete	**Low**
No privacy notice	Display Privacy Notice at registration	**Low**

Security Measures Implemented:

- MD5 Encryption - Passwords are encrypted and stored.
- HTTPS - The whole data transmission is encrypted.
- Access Control - Users have access to their own data only.
- Audit Logs - All actions are recorded
- Backups - Database will be backed up on a daily basis, after 24 hours.

Step 7: Sign Off and Record Outcomes

Item	Name/Position	Date
Measures approved by	Aayush Bhandari / Team Leader	17/04/2026
Residual risks approved by	Aayush Bhandari / Team Leader	17/04/2026
Supervisor advice provided	Mr Syed Altaf / Lecturer	17/04/2026

12.1.1 TEST & IMPLEMENTATION PLAN

Test Plan

Test Type	Tested
Unit Testing	Individual functions (login, add property, save bond)
Integration Testing	Frontend to backend connection
Security Testing	MD5 encryption, HTTPS, SQL injection
Performance Testing	Page load (<3 sec), API response (<500ms)
User Testing	All features work as expected

Some of Test Cases we performed:

TC ID	Test	Expected Result
TC01	Register with valid email	Account created
TC02	Register with existing email	Error message
TC03	Login with correct password	Dashboard opens
TC04	Login with wrong password	Error message
TC05	Add new property	Property saved

TC ID	Test	Expected Result
TC06	Record bond amount	Bond saved

Testing Tools

Tool	Purpose
Postman	API testing
Jest	Unit testing
Browser DevTools	Debugging
Manual Testing	User flow testing

Implementation Plan

Phases

Phase	Weeks	Activities	Assignees
1	Week 3-5	Requirements & Design	All
2	Week 5-7	Backend Development	Dadhi + Aayush
3	Week 8-10	Frontend Development	Linh + Asim
4	Week 10	Integration	All

Phase	Weeks	Activities	Assignees
5	Week 11	Testing	Aayush
6	Week 12	Deployment & Demo	Shubham + All

Deployment Steps

Step	Action	Tool
1	Push code to GitHub	Git
2	Deploy frontend	Vercel
3	Deploy backend	Railway
4	Setup database	PostgreSQL on Railway
5	Configure environment variables	Railway Dashboard
6	Test live application	Browser
7	Monitor logs	Vercel + Railway

13.1.1 RESPONSE TO FEEDBACK

No.	Feedback Received	Response / Action Taken
1	Tasks not allocated correctly on GitHub	All tasks reassigned based on team roles. Created 5 milestones (M1-M5) with proper assignees.
2	Milestones not created	Created M1: Project Initiation, M2: Requirements & SRS, M3: System Design, M4: Development M5: Testing and deployment
3	Slow project progress (only 5% completed)	Team notified immediately. Daily standup meetings implemented. All members completed their assigned sections.
4	Missing UI tasks	UI tasks assigned to Linh and Asim. Created wireframes (Login, Register, Dashboard, Properties, Bonds, Inspections, Documents, Profile).
5	Missing DFD Level 2 diagrams	DFD Level 2 created for all 6 processes (Authentication, Property, Bond, Inspection, Document, Dashboard).
6	FRs and NFRs incomplete	Completed 90 Functional Requirements and 25 Non-Functional Requirements.
8	ERD missing Crow's foot notation	Redesigned ERD with proper Crow's foot notation showing 1 to many relationships.
9	Gantt chart not provided	Created Gantt chart showing 12-week timeline with all phases and assignees.
10	Missing data encryption details	Added MD5 encryption for passwords and HTTPS for data transmission in Security section.

14.1.1 REFERENCE

- Kent Institute Australia. (2026). *CPRO306 Capstone Project Assessment Brief*. Melbourne, VIC: Kent Institute Australia.
- Skillup Labs. (2026). *Rental Bond Tracker Project Proposal*. Melbourne, VIC: Skillup Labs.
- Office of the Australian Information Commissioner (OAIC). (2023). *Australian Privacy Principles (APPs)*. Retrieved from https://www.oaic.gov.au/privacy/australian-privacy-principles (Original work published 1988).
- PostgreSQL Global Development Group. (2026). *PostgreSQL 18.3 Documentation*. Retrieved from https://www.postgresql.org/docs/18/.
- Meta Open Inc. (2026). *React 19.2 User Guide*. Retrieved from https://react.dev/.
- OpenJS Foundation. (2026). *Node.js 24 API Reference*. Retrieved from https://nodejs.org/docs/latest-v24.x/api/.
- OpenJS Foundation. (2026). *Express 5.x API Reference*. Retrieved from https://expressjs.com/en/5x/api.html/.
- diagrams.net. (2026). *draw.io User Manual*. Retrieved from https://www.diagrams.net/doc/.
- Australian Government. (2024). *Residential Tenancies Act 1997 (Vic)*. Retrieved from https://www.legislation.vic.gov.au/.
- Pressman, R. S. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). New York, NY: McGraw-Hill Education.

15.1.1 APPENDIX

Appendix A: Glossary of Terms

Term	Definition
API	Application Programming Interface - Connects frontend to backend
CRUD	Create, Read, Update, Delete - Basic database operations

Term	Definition
DFD	Data Flow Diagram - Shows how data moves through system
DPIA	Data Protection Impact Assessment - Privacy risk analysis
ERD	Entity Relationship Diagram - Shows database tables and relationships
FK	Foreign Key - References primary key in another table
FR	Functional Requirement - What the system must do
HTTPS	Hypertext Transfer Protocol Secure - Encrypted data transmission
JWT	JSON Web Token - Secure user authentication
MD5	Message Digest Algorithm 5 - Password hashing method
NFR	Non-Functional Requirement - System quality attributes
PK	Primary Key - Unique identifier for a table row
RBAC	Role-Based Access Control - User permission management
SQL	Structured Query Language - Database queries
SRS	Software Requirements Specification - This document
UI	User Interface - What users see and interact with
UX	User Experience - How users interact with the system

Term	Definition
WBS	Work Breakdown Structure - Project task breakdown

Appendix B: Technology Stack Summary

Category	Technology	Version	Purpose
Frontend	React	18.x	UI Framework
Frontend Framework	Next.js	14.x	Routing & SSR
Styling	TailwindCSS	3.x	Responsive Design
Backend	Node.js	20.x	JavaScript Runtime
Backend Framework	Express.js	4.x	API Development
Database	PostgreSQL	15.x	Relational Database
Authentication	JWT + MD5	-	Security
Version Control	Git / GitHub	-	Source Code
Frontend Hosting	Vercel	-	Deployment
Backend Hosting	Railway	-	Deployment
File Storage	AWS S3	-	Document Storage

Category	Technology	Version	Purpose
Wireframing	Balsamiq Cloud	-	UI Design
Diagramming	draw.io	-	DFD, ERD
API Testing	Postman	-	Testing
Unit Testing	Jest	29.x	Testing

Appendix C: Team Sign-off

Role	Name	Signature	Date
Team Leader + Backend	Aayush Bhandari	Aayush	19/04/2026
Database Designer	Dadhi Ram Poudel	Dadhi	19/04/2026
Business Analyst	Sujan Lamichhane	Sujan	19/04/2026
Front-End Developer	Nguyen Nhat Linh Dinh	Linh	19/04/2026
Front-End + UI	Asim Bhattarai	Asim	19/04/2026
QA + AI	Shubham Shrestha	Shubham	19/04/2026
Project Supervisor	Mr Syed Altaf		

Appendix D: Document Version History

Version	Date	Author	Changes
0.1	01/04/2026	Aayush	Initial draft
0.5	10/04/2026	All	Added FRs, NFRs, DFDs
1.0	19/04/2026	Aayush	Final SRS report

Appendix E: List of Diagrams

Diagram	Section	Tool
Organization Chart	1.1.6	draw.io
System Architecture	3.1.1	draw.io
Technology Stack	3.1.3	draw.io
Context Diagram (DFD 0)	6.1.1	draw.io
DFD Level 1	7.1.1	draw.io
DFD Level 2	7.1.2	draw.io
Use Case Diagram	5.1.1	draw.io
Sequence Diagram	8.1.1	draw.io
ERD	9.1.1	draw.io
Class Diagram	10.1.1	draw.io

Diagram	Section	Tool
Gantt Chart	1.1.11	Excel

Appendix F: UI Wireframes (Balsamiq Cloud)

Screen	Description
Login Page	Email and password fields
Register Page	Full name, email, password, phone
Dashboard	Stats cards, quick actions
Properties List	View all properties
Add Property Form	Address, landlord, agent details
Bonds List	Bond records with status
Add Bond Form	Amount, date, reference
Inspections List	Inspection history
Add Inspection Form	Date, type, notes, photos
Documents List	Uploaded files
Profile Page	User information